

07

TESTING / INTEGRATION / DELIVERY

KURSinHALTE UND TERMINE

Kursinhalte

Data Science	1
Statistische Grundlagen	2
Softwareentwicklung	3
Analysemethoden, Clustering und Assoziationen	4
Grundlagen Künstlicher Intelligenz (KI)	5
Maschinelles Lernen, Klassifikation und Künstliche Neuronale Netze (KNN)	6
Testing / Integration / Delivery	7
Anwendungen Künstlicher Intelligenz (KI)	8
Zusammenfassung / Fragen / Klausurvorbereitung	9

Termine

#	Wochentag	Datum	von - bis	Räume
1	Freitag	04.04.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
2	Freitag	11.04.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.27 Bothfeld
3	Freitag	25.04.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
4	Freitag	16.05.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
5	Freitag	23.05.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
6	Freitag	06.06.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
7	Donnerstag	26.06.2025	13:15 - 16:30	HAN - Schiffgraben 49-51 - 0.07 Marktkirche
8	Freitag	04.07.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt
9	Freitag	11.07.2025	09:00 - 12:15	HAN - Schiffgraben 49-51 - 1.24 Südstadt

Lernziele

Nach der Bearbeitung dieser Lektion werdet ihr wissen, ...

- Warum das **Testen** von **Applikationen** wichtig ist
- Was man unter einem **Unit-Test** versteht
- Was man unter einem **Integrations-Test** versteht
- Was man unter **“Test-Driven Development” (TDD)** versteht
- Besonderheiten beim **Testen** in **agilen Projekten**
- Wie man ein **lokales Modell** in eine verteilte **Produktionsumgebung** bringt
- Was man unter **„Continuous Integration (CI)“** / „Kontinuierliche Integration“ versteht
- Was man unter **„Continuous Delivery (CD)“** / „Kontinuierliche Auslieferung“ versteht

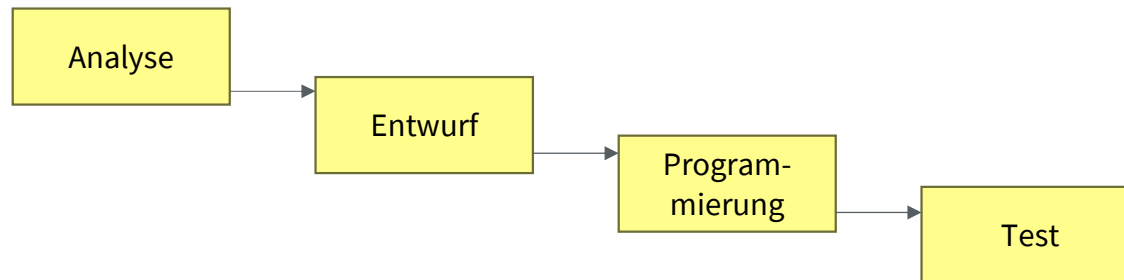


SOFTWAREENTWICKLUNG

KLASSISCHE PROJEKTMANAGEMENT-ANSÄTZE

Beim klassischen Projektmanagement in der Softwareentwicklung werden die verschiedenen Entwicklungsaktivitäten mittels Phasen nacheinander durchlaufen und von entsprechenden Experten für die jeweilige Phase umgesetzt. Zwischen den Phasen finden Übergaben statt und es wird angestrebt Rücksprünge zu vorherigen Phasen möglichst zu vermeiden.

Ein extremes Beispiel mit streng sequentiellen Phasen ist das „**Wasserfall-Modell**“



Quelle: Vgl. Laudon et. al, 2016, S 890.

Die Phasen können weiter untergliedert werden (bspw. Entwurf in Software- und Systementwurfsphase).

Testen

“Der umfassende und sorgfältige **Prozess**, der **feststellt**, ob das System **unter bekannten Bedingungen** die **gewünschten Ergebnisse** erzielt.”

„Testen kann nicht zeigen, dass Software keine Fehler enthält, aber maßgeblich das Vertrauen in die korrekte Funktion erhöhen.“

Zeit fürs Testen wird bei der Projektplanung häufig unterschätzt!
Testaktivitäten sind zeitaufwändig!
Hohes Risiko der zeitlichen und kostenmäßigen Unterschätzung.

Testdaten müssen erhoben, Ergebnisse überprüft, bei Bedarf Änderungen am System vorgenommen oder neu entworfen werden.

SOFTWAREENTWICKLUNG

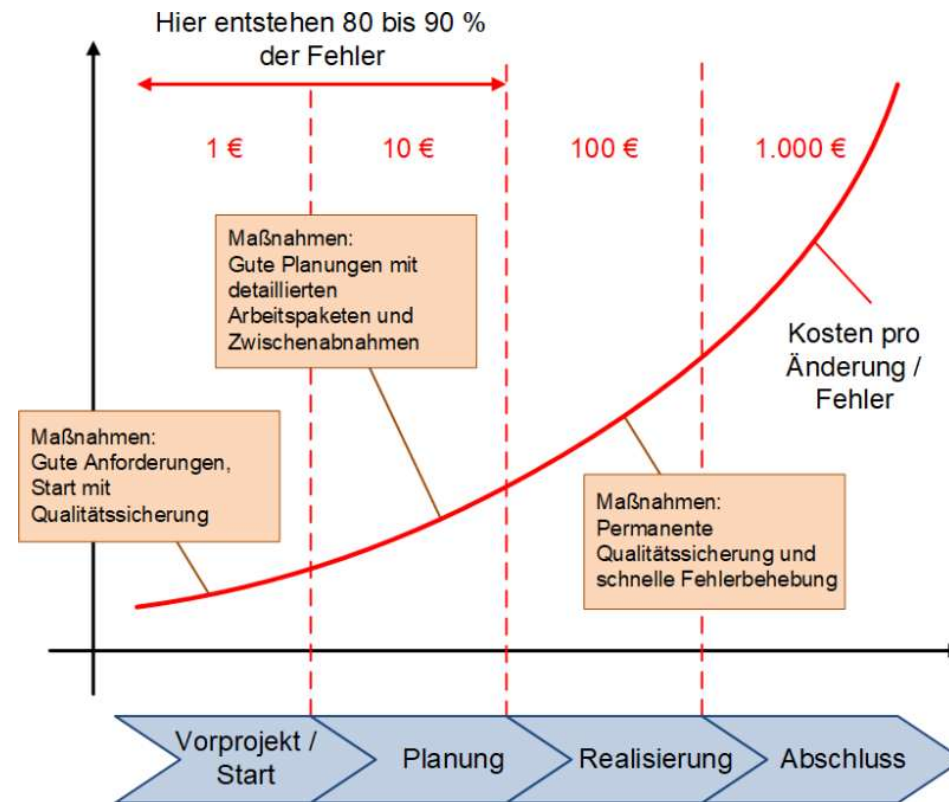
FEHLERKOSTEN 10ER REGEL

Die 10er-Regel der Fehlerkosten (engl. *Rule of 10 of the error costs*) beschreibt den Sachverhalt, dass die Behebung eines Fehlers teurer wird, je später er gefunden wird.

Auch wenn es 10er-Regel heißt, so kann der Kostenzuwachs / -faktor zwischen den Entwicklungsstufen auch kleiner oder größer als 10 sein.

Zentrale Aussage: Wenn Fehler nicht frühzeitig vermieden oder erkannt werden, so wird die Behebung später teuer

„Die Folgerung aus der 10er-Regel ist einfach: Es muss das Ziel einer Produktentwicklung sein, so zu entwickeln, dass möglichst wenig Fehler in den frühen Entwicklungsphasen entstehen und diese zudem möglichst frühzeitig behoben und erkannt werden.“

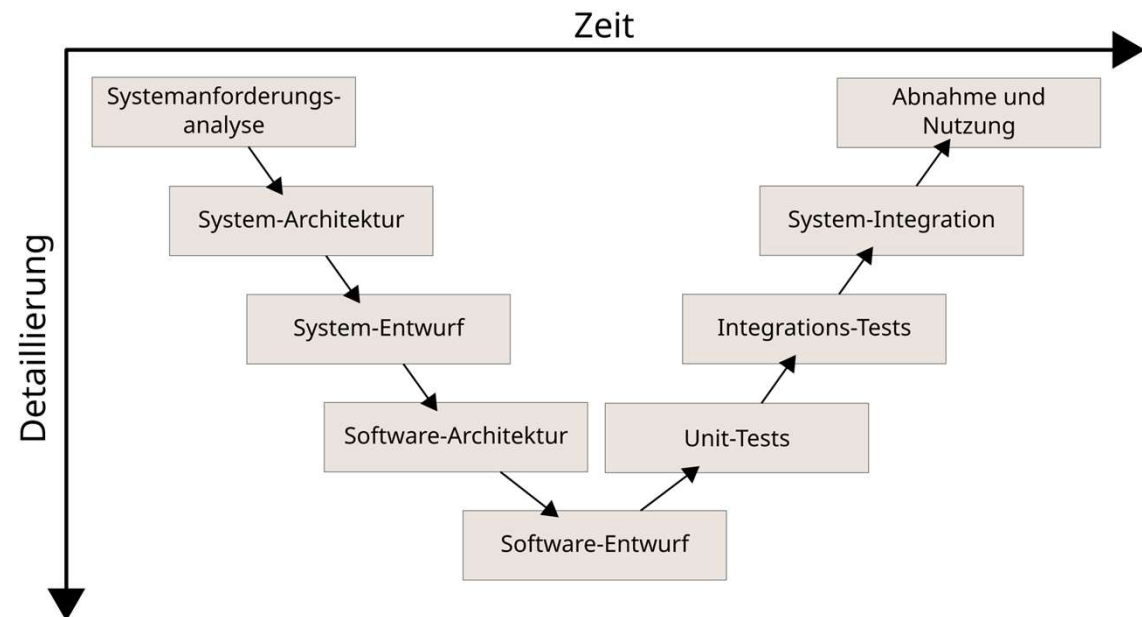


Quelle: <https://www.peterjohann-consulting.de/10er-regel-der-fehlerkosten/>

Quelle: <https://www.peterjohann-consulting.de/10er-regel-der-fehlerkosten/>

TESTSTUFEN V-MODELL

„Das **V-Modell** beschreibt ein sequentielles Vorgehensmodell der Software- und Systementwicklung, das Wert auf **die frühe Planung** und **spätere Verifikation und Validierung** legt. Die linke Seite des V steht für die **Spezifikation**, die rechte für die **Teststufen**.“ (Balzert, 2001)



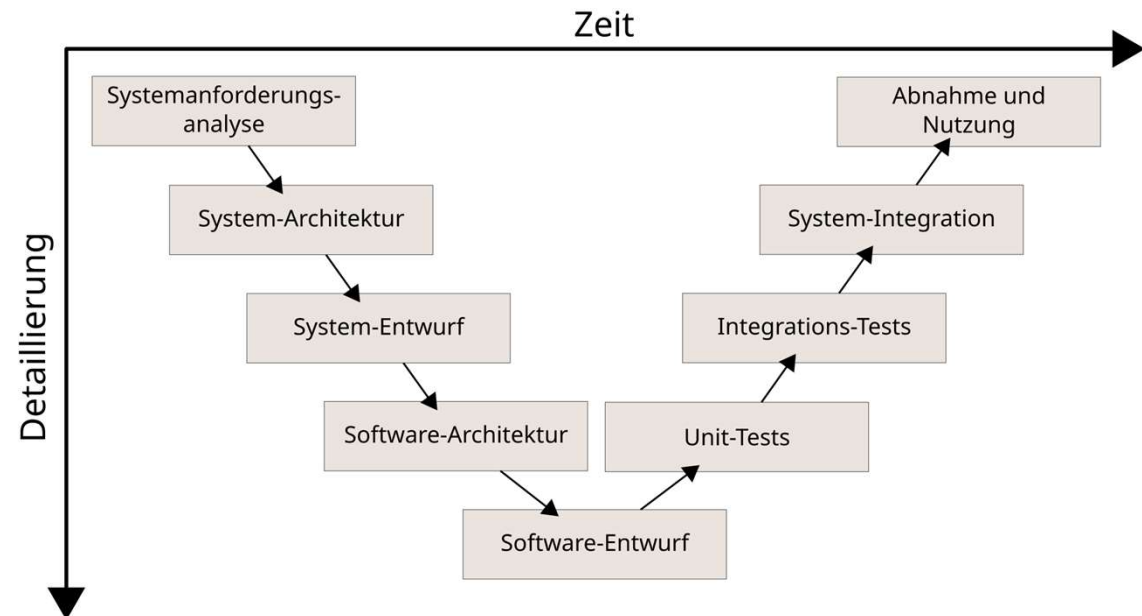
Quelle: <https://de.wikipedia.org/wiki/V-Modell>

TESTSTUFEN V-MODELL

Modultest / Unit-Test

“Der Prozess, jedes Programm (bzw. jedes Modul) im System **seperat** zu **testen**.”

- Modul ist Einheit eines Systems die ohne Integration mit anderen Einheiten simple Aufgaben erfüllt
- Tests sind immer nur Stichproben
- Ziel sollte sein, möglichst viele und schwerwiegende Fehler zu finden
 - Typischerweise Aufgabe der Entwickler



Quelle: <https://de.wikipedia.org/wiki/V-Modell>

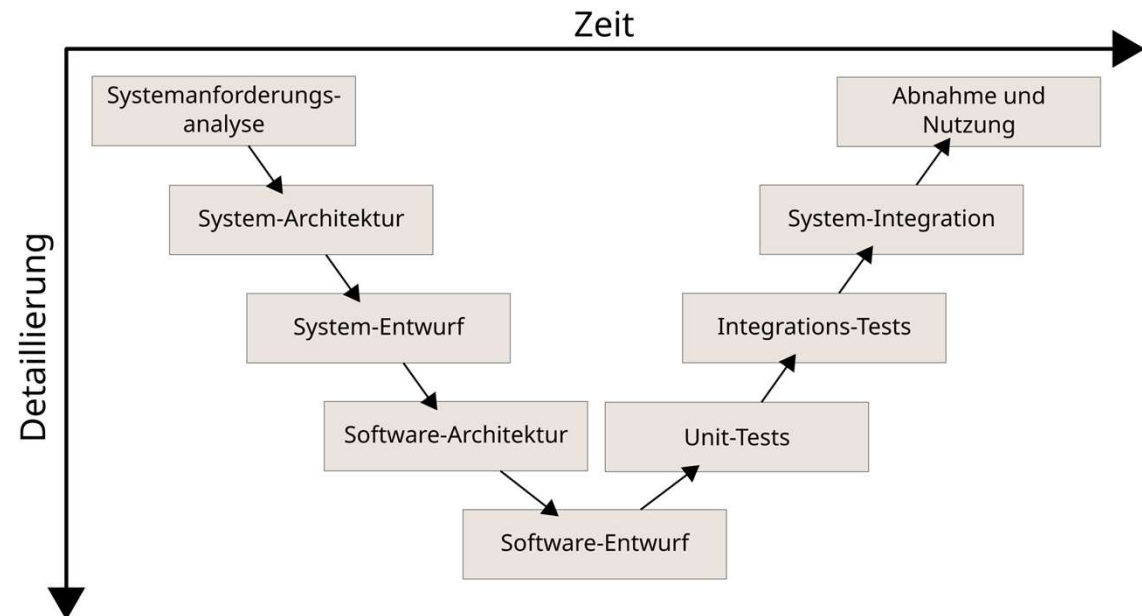
TESTSTUFEN V-MODELL

Integrationstest / Systemtest

“Testen, ob das Informationssystem als Ganzes funktioniert, um festzustellen, ob die einzelnen Module wie vorgesehen zusammenarbeiten.”

- Feststellen, ob die Module wie vorgesehen zusammenarbeiten
- Laufzeitverhalten, Kapazität für Datenspeicher und Umgang mit Spitzenlasten können Themen sein
- Wichtig ist die Integration der Module, da sich manche Fehler erst im Zusammenspiel zeigen

Quellen: Laudon et al., Wirtschaftsinformatik, 2016, S.883f.



TESTSTUFEN V-MODELL

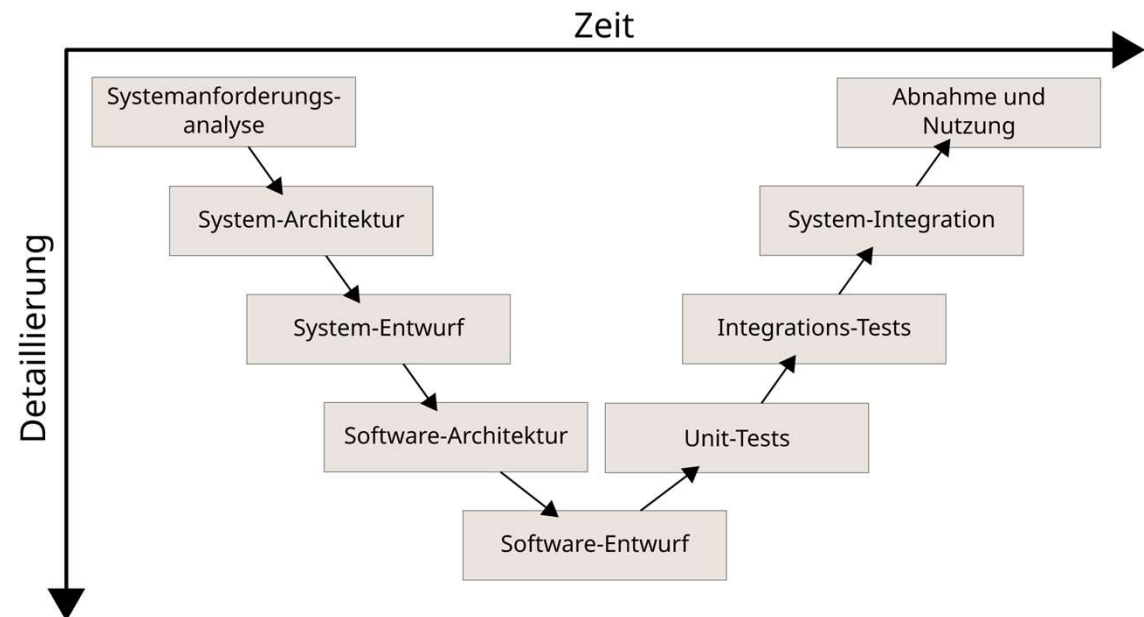
Akzeptanztest / Abnahmetest

“Verhilft zu einer Einschätzung, ob das System für den Einsatz in einer Produktionsumgebung bereit ist.”

Benutzer testen das System auf die Anforderungen

Soll helfen die Wahrscheinlichkeit einzuschätzen, ob das System für die Produktionsumgebung bereit ist

– Akzeptanztests sind meistens nicht technisch geschrieben und können hohen manuellen Aufwand aufweisen



Quelle: <https://de.wikipedia.org/wiki/V-Modell>

TESTPYRAMIDE

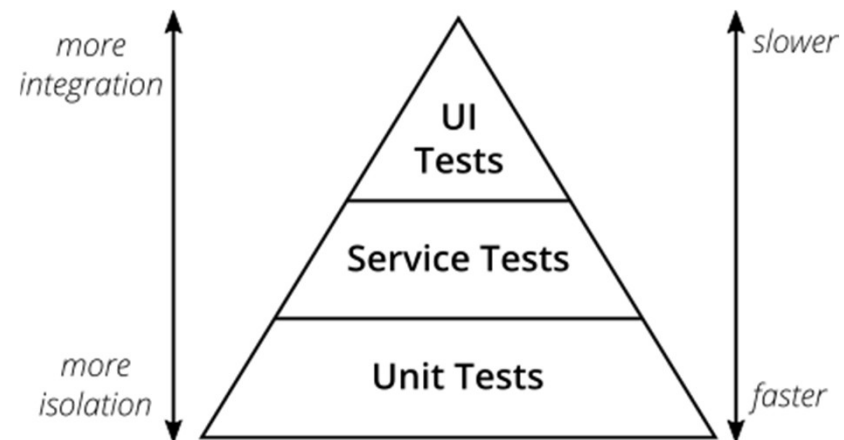
Testpyramide nach Mike Cohn (2009)

Daumenregel:

1. Erstelle Tests mit unterschiedlicher Granularität
2. Je höher in der Pyramide, umso weniger Tests

Ziel der Pyramide

- **Mehr Tests auf tieferen Ebenen (Unit)**, da sie schnell und zuverlässig sind
- **Weniger auf hohen Ebenen (E2E/UI)**, da sie instabiler und langsamer sind
- So entsteht ein **stabiles, kosteneffizientes Testsystem**



Quelle: <https://martinfowler.com/articles/practical-test-pyramid.html>

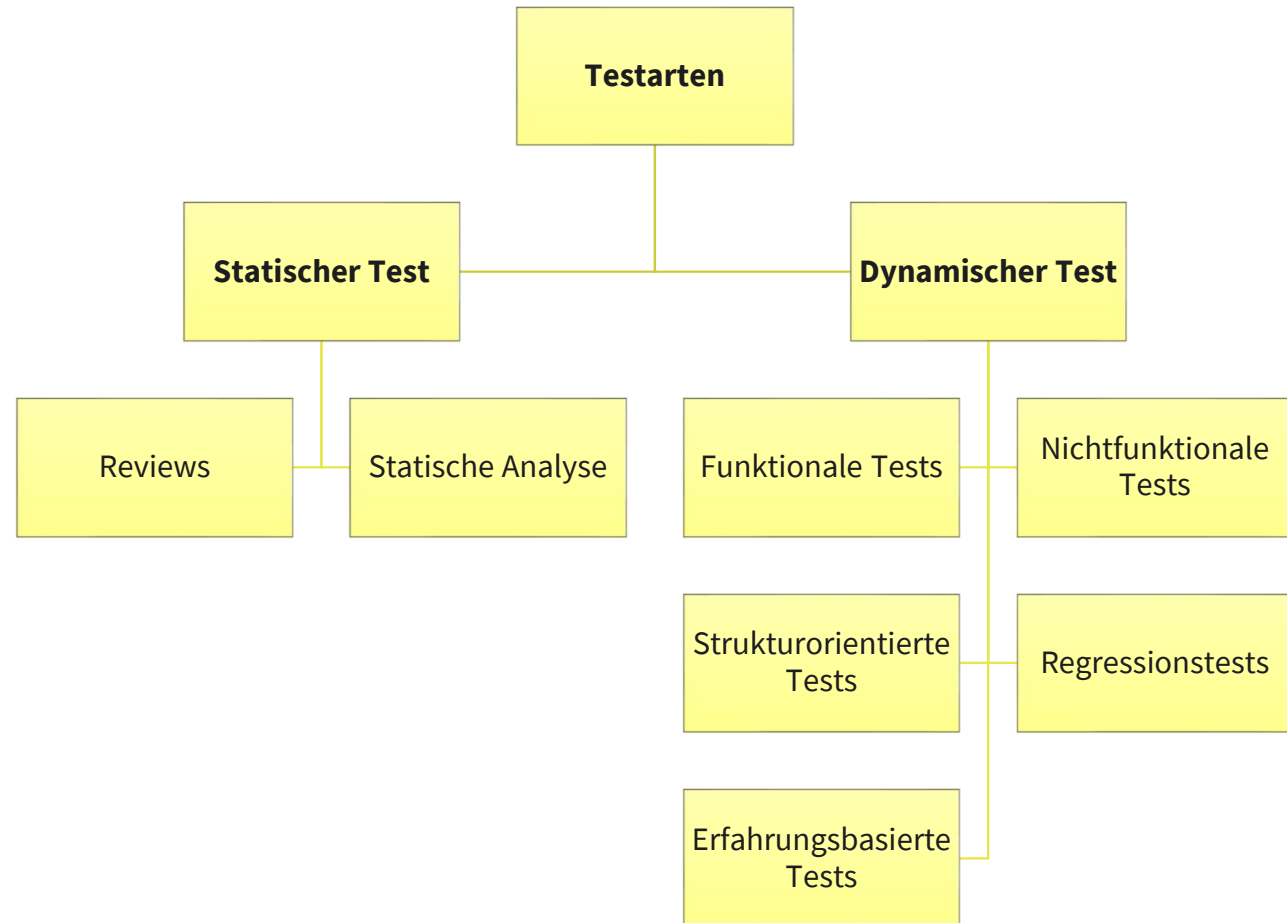
Quelle: <https://martinfowler.com/articles/practical-test-pyramid.html>

TESTARTEN

Eine **Testart** ist eine Menge von Aktivitäten die verwendet werden, um ein System auf Qualitätsmerkmale zu überprüfen.

Jede **Teststufe** kann mehrere **Testarten** beinhalten.

Die **Testarten** bedienen sich unterschiedlicher **Methodiken**.

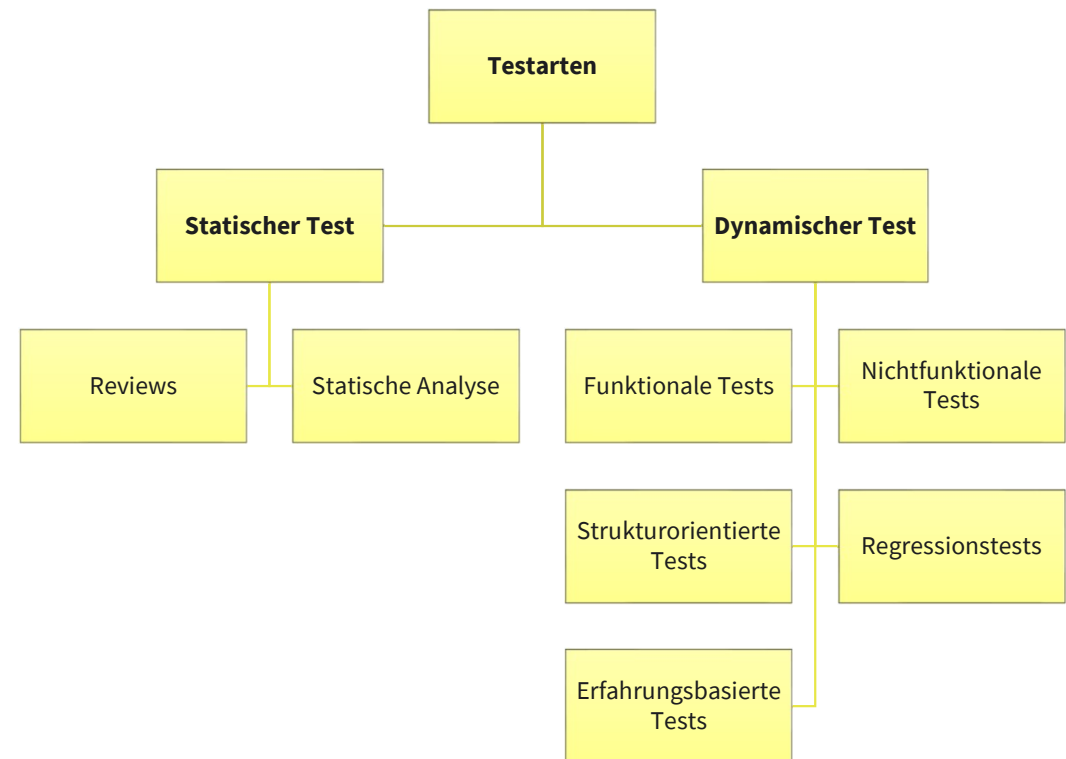


Quelle: <https://martinfowler.com/articles/practical-test-pyramid.html>

TESTARTEN

Statischer Test

Testen einer Komponente oder eines Systems auf Anforderungs- und Implementierungsebene ohne Ausführung der Software. Prüfgegenstand sind der Programmcode und alle Dokumente, die während eines Projektes erstellt werden. Das Ziel ist Aufdecken von Fehlerzuständen, bspw. mittels **Reviews** oder **statischer Codeanalyse**.



Quelle: <https://martinfowler.com/articles/practical-test-pyramid.html>

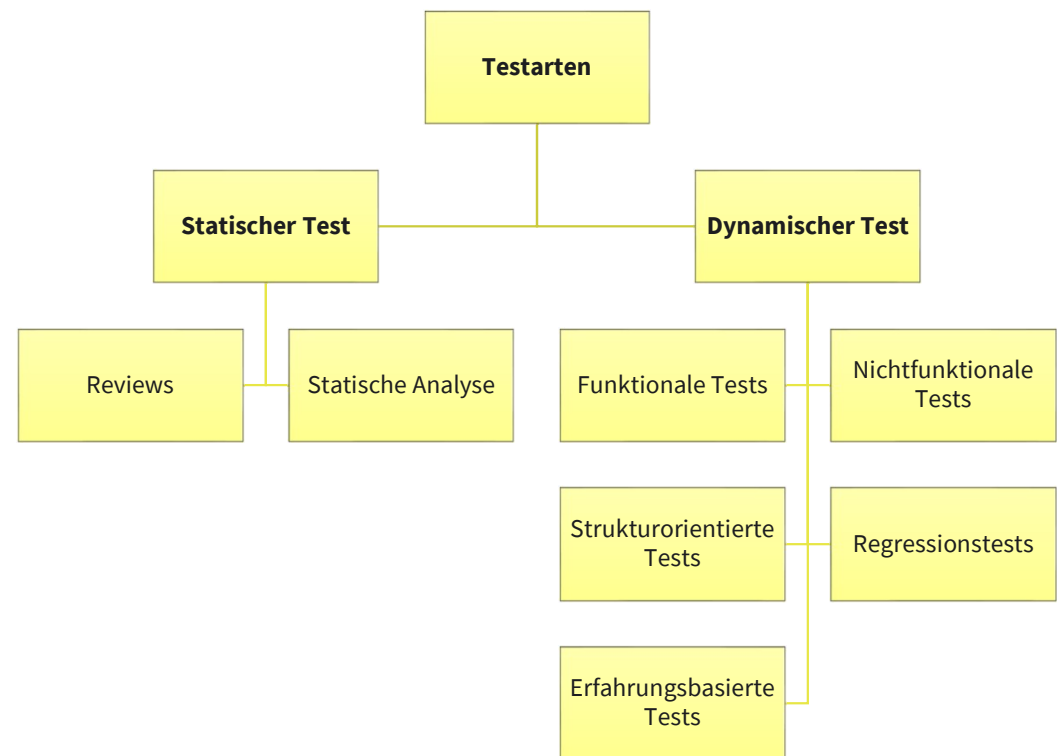
TESTARTEN

Reviews

Bezeichnung für Prüfung von Dokumenten als auch für statische Prüfverfahren, die von Personen durchgeführt werden. Mittel zur Sicherung der Qualität, welches idealerweise so früh wie möglich nach der Fertigstellung durchgeführt wird. Die Person, die ein Review durchführt, sollte über entsprechendes Fachwissen und Kenntnisse verfügen.

Unterschiedliche Review-Arten:

- **Walkthrough**
- **Inspektion**
- **Technisches Review**
- **Informelles Review (auch Peer Review)**



Quelle: <https://martinfowler.com/articles/practical-test-pyramid.html>

Statischer Test

Review-Arten

Walkthrough

- Autor präsentiert den Gutachtern
- Verteilung des Wissens über das Prüfobjekt auf alle Beteiligte
- Diskussion von Lösungsalternativen
- Prüfung der Einhaltung der Spezifikationen und Standards
- für kleine Entwicklungsteams bis zu 5 Personen geeignet
- verursacht wenig Aufwand

Inspektion

- Formale Vorgehensweise mit klarer Rollenverteilung
- Gutachter prüfen Testobjekt nach festgelegten Kriterien (Checklisten, Metriken)
- Reviewfähigkeit des Prüfobjektes wird anhand von festgelegten Eingangsbedingungen überprüft
- Formaler Inspektionsbericht mit Befundliste wird erstellt
- Konkrete Ziele der Prüfung werden bei der Planung festgelegt
- Aufwand ist aufgrund des hohen Formalitätsgrades relativ groß

Technisches Review

- Überprüfung ob das Prüfobjekt mit der Spezifikation übereinstimmt, für den Einsatz geeignet ist und relevante Standards einhält
- Wird von Fachexperten (nach Möglichkeit projektextern) durchgeführt
- Autor muss nicht dabei sein
- Aufwand zur Festlegung der Prüfkriterien

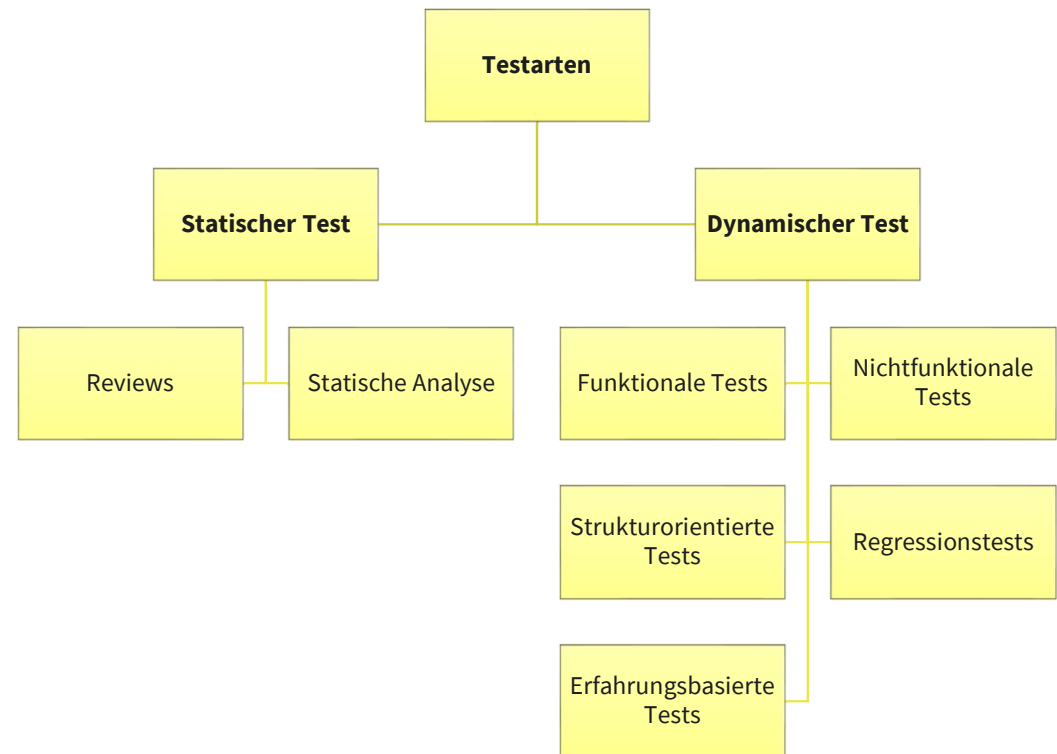
Informelles Review (auch Peer Review)

- Abgeschwächte Version eines Reviews mit geringem Aufwand
- Folgt gleicher Vorgehensweise nur in vereinfachter Form
- Autor selbst beruft das Review ein
- Häufig entsteht eine Liste mit Anmerkungen von den Gutachtern

TESTARTEN

Statische Analyse

- Testen von Prüfobjekten, die eine formelle Struktur haben (bspw. technische Anforderungen, Softwarearchitektur, Softwareentwurf, Teile der Implementierung)
- White-Box-Test-Verfahren (der innere Aufbau des Testobjekts wird während des Tests beobachtet)
- werkzeugbasierte Analysemethode
- wird meist beim Modul- und Integrationstest genutzt, um Programmierkonventionen und Schnittstellendefinitionen zu testen
- Kann sinnvoller Weise im Vorfeld zu einem Review durchgeführt werden, so dass die Arbeit im Review weniger wird
- Compiler können auf Einhaltung von Syntax prüfen
- Analysatoren können Programmcode auf Einhaltung von Konventionen und Standards der jeweiligen Programmiersprache prüfen
- Ermittlung von Messgrößen und Metriken zu Qualität
- Benötigt kaum Zeit und Ressourcen

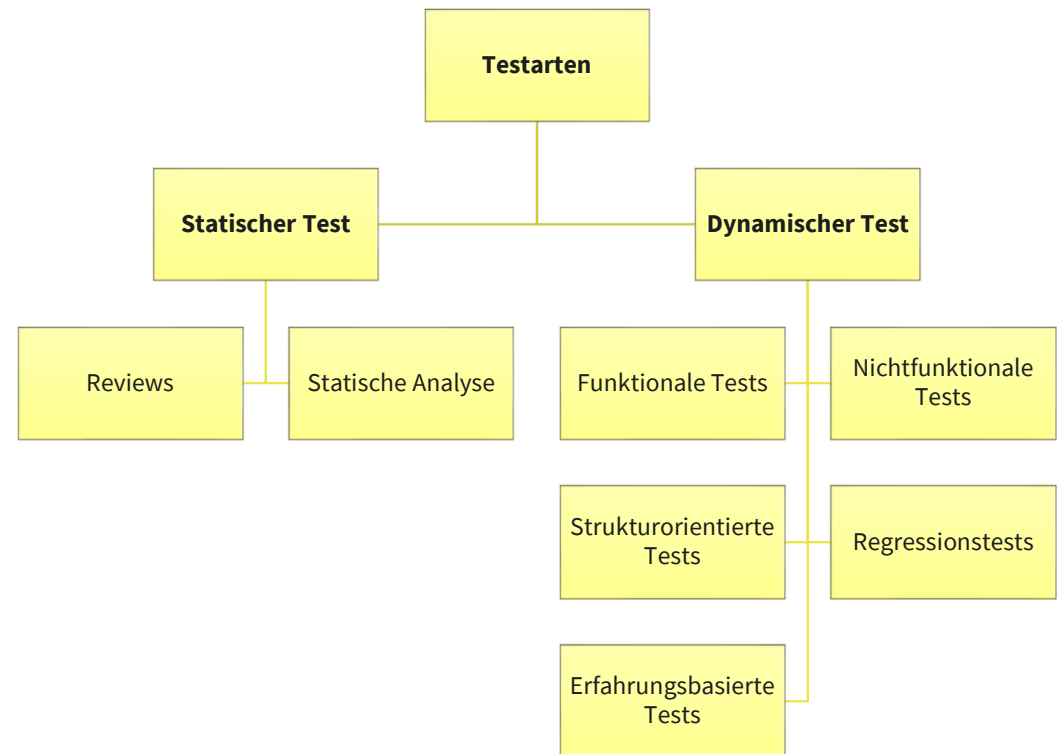


Quelle: <https://martinfowler.com/articles/practical-test-pyramid.html>

TESTARTEN

Dynamischer Test

- Testverfahren bei denen das Testobjekt ausgeführt wird, um die korrekte Funktionsweise zu prüfen
- Können erst durchgeführt werden wenn ein ablauffähiges Programm vorliegt
- Zur Durchführung werden häufig Mockups benötigt, die (Teil-)Programme oder Daten simulieren
- Ziel ist der Nachweise, dass die Anforderungen erfüllt werden und die Aufdeckung von Fehlerwirkungen
- Um möglichst viele Anforderungen und Fehlerwirkungen abzudecken, ist die systematische Auswahl von Testfällen sehr wichtig
- Es gibt Black-Box-Tests und Whitebox-Tests



Quelle: <https://martinfowler.com/articles/practical-test-pyramid.html>

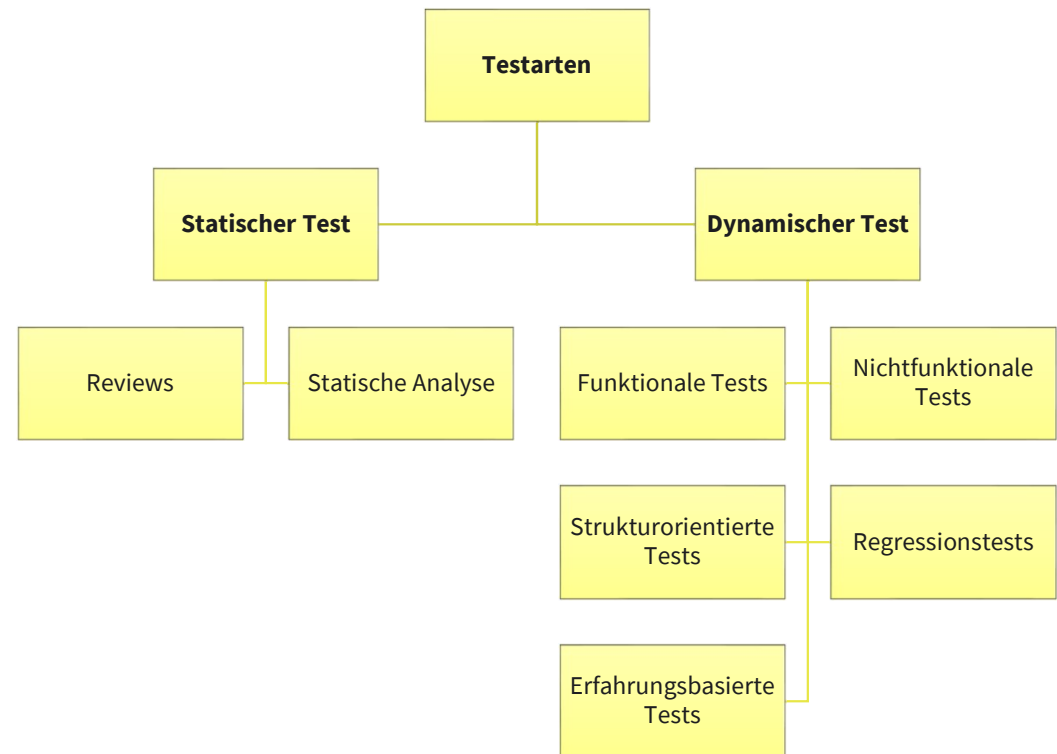
TESTARTEN

Funktionale Tests

- Überprüfen Attribute der funktionalen Anforderungen
- Umfasst Testmethoden, die das von außen sichtbare Ein-/Ausgabeverhalten eines Testobjekts überprüfen
- Ziel ist die Überprüfung, ob die Anforderungen erfüllt werden
- Häufig Black-Box-Verfahren, die auf einen Input den erwarteten Output prüfen
- Werden ab der Stufe Systemtest eingesetzt

Nichtfunktionale Tests

- Werden ab der Stufe Systemtest eingesetzt
- Zuerst nichtfunktionale Anforderungen (bspw. Zu Performance, Usability, Interoperabilität, Sicherheit) in messbare Größen verfeinern
- Überprüfung inwiefern die definierten Zielwerte eingehalten werden



Quelle: <https://martinfowler.com/articles/practical-test-pyramid.html>

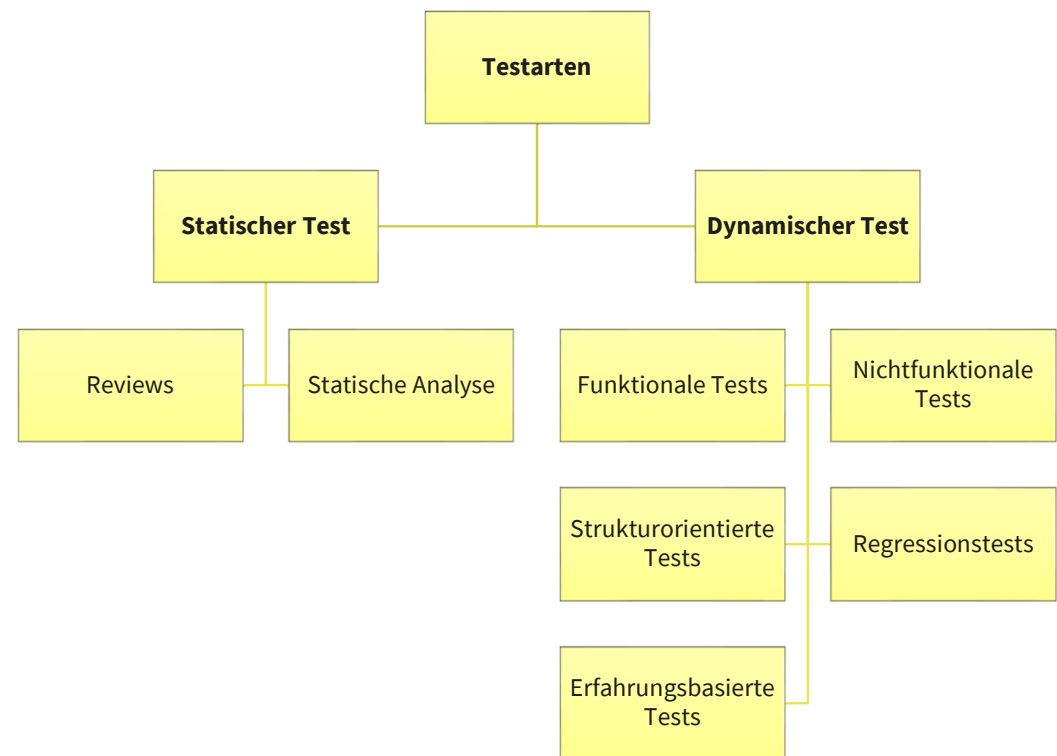
TESTARTEN

Strukturorientierte Tests

- Untersuchung der internen Struktur der zu testenden Software
- Testen des Sourcecode als auch Testen der Schnittstellen und das Zusammenspiel der Komponenten
- Kommt in den Teststufen Komponententest und Integrationstest zum Einsatz

Regressionstests

- Erneutes Testen eines bereits getesteten (Teil-)Programms nach deren Änderung
- Nachweisen, dass durch die neuen Änderungen keine neuen Fehlerzustände eingebaut oder freigelegt wurden
- Wichtig ist die Wiederholbarkeit der Tests
- Bevorzugte Kandidaten für die Testautomatisierung

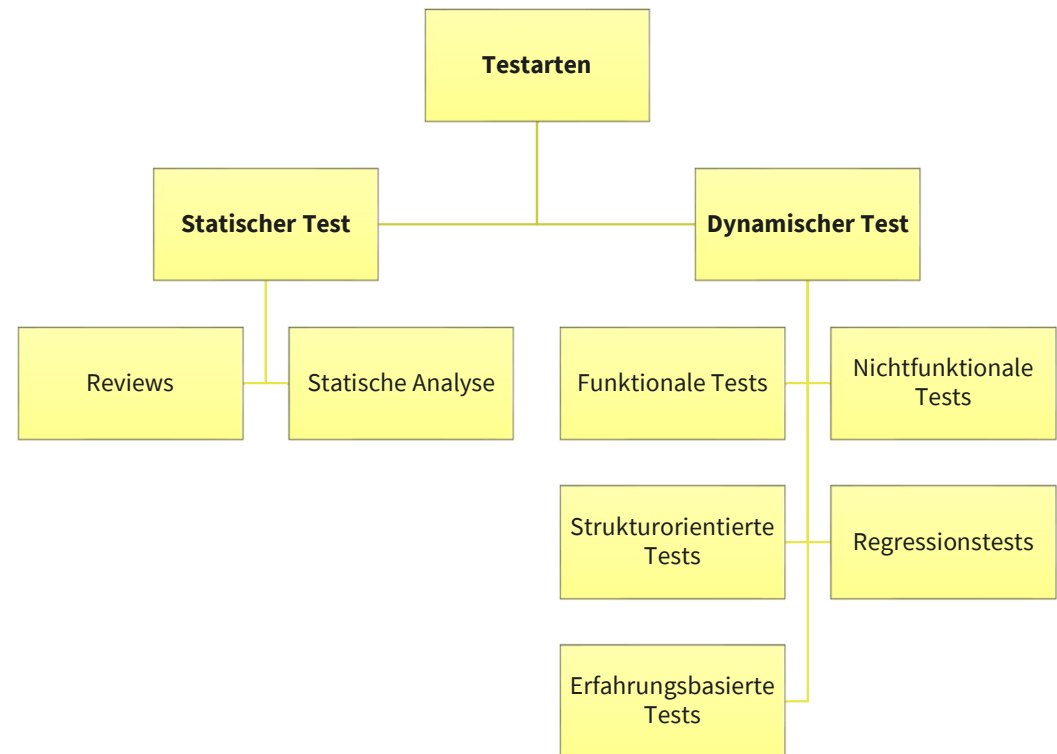


Quelle: <https://martinfowler.com/articles/practical-test-pyramid.html>

TESTARTEN

Erfahrungsbasierte Tests

- Dienen als Ergänzung oder Vorabtesten für strukturierte Verfahren
- Basieren auf der Erfahrung / Vermutung / Intuition wo in der Vergangenheit Fehler aufgetreten sind
- Können Fehlerwirkungen aufdecken, die von anderen Testarten übersehen werden können
- Keine methodische Vorgehensweise
- Voraussetzung ist eine vernünftige Dokumentation
- Findet sich in allen Teststufen wieder



Quelle: <https://martinfowler.com/articles/practical-test-pyramid.html>

TEST- DRIVEN DEVELOPMENT (TDD)

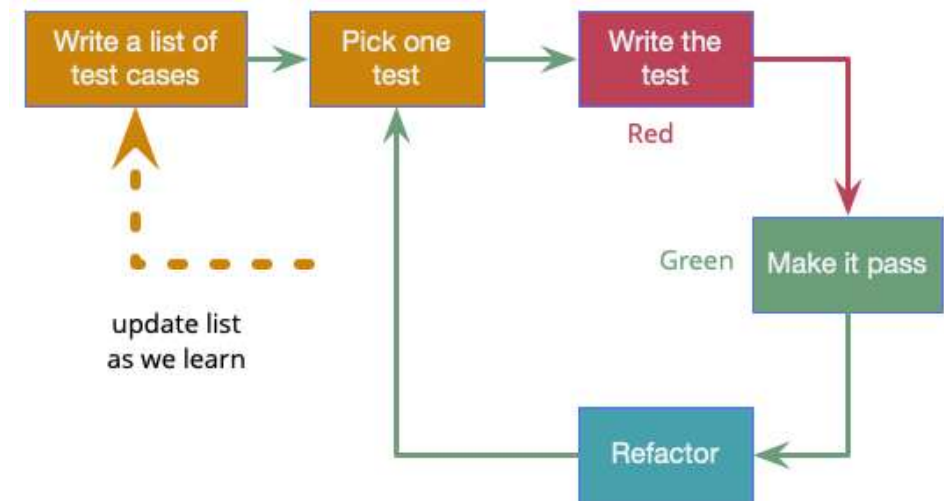
„**Test-Driven Development** is a way of managing fear during programming. It works by **writing an automated test before writing the code** to make the test pass.“

(Kent Beck, 2002)

“**Test-Driven Development (TDD)** is a technique for building software that guides software development by writing tests. It was developed by Kent Beck in the late 1990's as part of Extreme Programming. In essence we follow three simple steps repeatedly:”

- Write a test for the next bit of functionality you want to add.
- Write the functional code until the test passes.
- Refactor both new and old code to make it well structured.

(Martin Fowler, 2024)

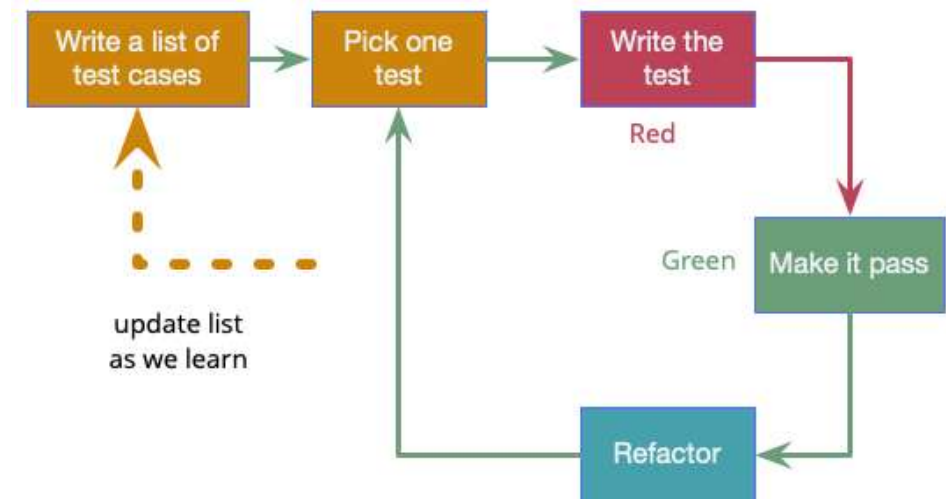


Quelle: Martin Fowler, <https://martinfowler.com/bliki/TestDrivenDevelopment.html>

TEST- DRIVEN DEVELOPMENT (TDD)

“... use the discipline of **Test-Driven Development (TDD)**. One of the central doctrines of this approach is to keep the system running at all times. In other words, using TDD, I am not allowed to make a change to the system that breaks that system. Every change I make must keep the system working as it worked before” (Robert C. Martin, 2009)

Ziel von TDD ist es, sauberen, verständlichen und gut getesteten Code zu schreiben. Es fördert Designqualität, Testabdeckung und frühes Feedback über Fehler.



Quelle: Martin Fowler, <https://martinfowler.com/bliki/TestDrivenDevelopment.html>

Lernziele



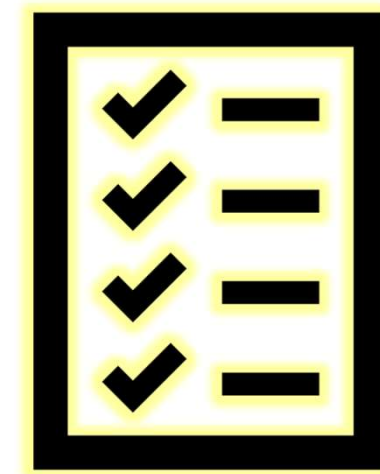
Nach der Bearbeitung dieser Lektion werdet ihr wissen, ...

- Warum das **Testen** von **Applikationen** wichtig ist
 - Was man unter einem **Unit-Test** versteht
 - Was man unter einem **Integrations-Test** versteht
 - Was man unter **“Test-Driven Development” (TDD)** versteht
 - **Besonderheiten beim Testen in agilen Projekten**
- Wie man ein **lokales Modell** in eine verteilte **Produktionsumgebung** bringt
 - Was man unter **„Continuous Integration (CI)“** / „Kontinuierliche Integration“ versteht
 - Was man unter **„Continuous Delivery (CD)“** / „Kontinuierliche Auslieferung“ versteht

TESTEN IN AGILEN PROJEKTEN

Testvorgehen

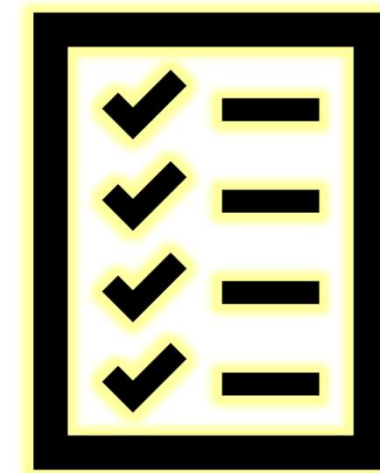
- Anspruch der agilen Vorgehensweise, Software mit hoher Qualität fortlaufend zu entwickeln
- Erreichung durch agile Vorgehensweisen und die dazugehörigen Ergebnistypen:
 - Wert- und risikogetriebene Priorisierung der Arbeitsumfänge => **Produktbacklog**
 - Iterative Entwicklungs-Vorgehen => **Sprints**
 - Frühe Feedbacks durch den Nutzer/Kunden => **Sprint Review als Retrospektive**
 - Hohe Automatisierung in der Produktentstehung und Integration => **automatisierte Tests** und **Continuous Integration**.
- In einem agilen Kontext gibt es keine explizit lange Testphase am Ende der Entwicklung, somit sind die **Test-Aktivitäten zum Großteil in den Sprints** einzuplanen.
- **Separate Testphase** wird auf die verbleibenden Tests minimiert (z.B. ergänzende explorative Abnahmetests).
- Notwendige **umfangreiche Tests** (z.B. im Rahmen einer Gesamtintegration) werden **separat zum Releasezyklus** eingeplant.



TESTEN IN AGILEN PROJEKTEN

Testkonzeption

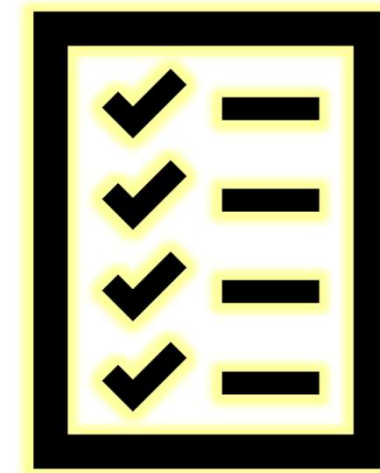
- Konzeptionelle Dokumente für das agile Projekt können in der agilen Methodik „on Demand“ entwickelt werden – z.B. vor dem ersten Sprint in der agilen **Pilotphase**.
- Bei **Veränderungen** sind konzeptionelle Dokumente **nach** jedem **Sprint** oder Release zu **aktualisieren**.
- Detailierung erfolgt als **eigene User Story** (bspw. Story zur Testautomatisierung oder der Bereitstellung von Testdaten).
- Die **inhaltliche Vollständigkeit** der einzelnen Konzeptelemente sollte **so früh wie möglich** erstellt werden.
- Die Teststrategie sollte von vornherein das Thema **Automatisierung von Tests** beinhalten
- Die zugehörigen **Ressourcen**, **Testwerkzeuge** und **Testumgebungen** sind einzuplanen und **bereitzustellen**.



TESTEN IN AGILEN PROJEKTEN

Agile Basisabsicherung im Test

- Eine Basisabsicherung soll sicherstellen, dass verschiedene **Testaktivitäten** aufeinander aufbauen, keine unnötige **Doppelabsicherung** entsteht und eine adäquate **Testabdeckung** erzielt wird.
- **Continuous Integration** ist eine wichtige Grundlage zur **Basisabsicherung** einer User Story.
- Folgende Elemente könnten zum Beispiel eine agile Basisabsicherung in Bezug auf Test darstellen:
 - **Statische Code Analyse**
 - **Automatisierte Unit Tests**
 - **Automatisierte Integrations- und Systemtests im Sinne Regressionstestset**
 - **Manuelle Systemtests**
 - **User Acceptance Test**
- In agilen Projekten werden die Entwicklungsphasen in iterativen Zyklen durchlaufen, weshalb ein hoher Automatisierungsgrad von Testfällen notwendig ist.

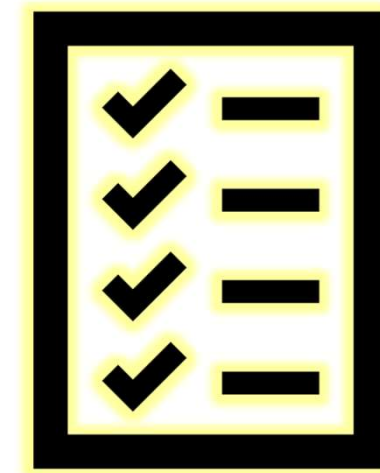


Definition of Done (DoD)

- Die **Definition of Done** legt fest, welche zuvor spezifizierten Kriterien erfüllt sein müssen, damit eine **User Story** als **abschlossen** gilt.

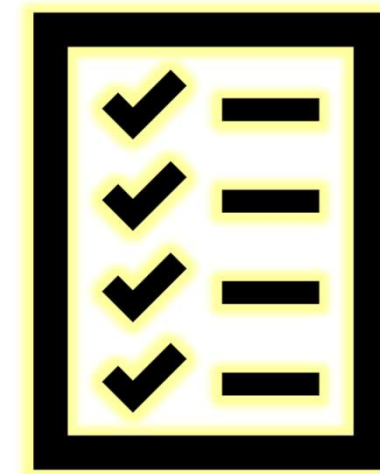
Beispielhafte Bestandteile einer Definition of Done:

- Source Code ist eingereviewt
- Produkt Inkrement ist eingereviewt
- Programmierrichtlinien (Coding Conventions) sind eingehalten, Statische Code-Analyse
- ist erfolgt und Code-Reviews wurden durchgeführt
- Test & QA-Anforderungen laut Minimalanforderungen der Basisabsicherung
- Entwickler- bzw. Unit-Tests sind durchgeführt und dokumentiert
- Regression-Tests (d.h. funktionsorientierte Szenarien-Tests) sind erfasst
- Testfälle für Akzeptanz-Tests sind in adäquaten Werkzeugen verfügbar
- Qualitätsvorgaben laut Testkonzept sind erfüllt
- Relevante Dokumentation ist erstellt / aktualisiert
- Testergebnisse sind nachvollziehbar dokumentiert
- Akzeptanzkriterien sind erfüllt



Definition of Done (DoD) Stufen

- In der Praxis bewährt es sich **gestufte Vorgehen** zu etablieren, in denen ausgehend von einer Definition of Ready (DoR) gestufte DoD zu definieren sind, die ein **LoD (Level of Done)** abbilden.
- Hier ein paar exemplarische Stufen:
 - **DoR:** Eine Story ist im Sinne der Anforderung ausreichend definiert (inkl. Akzeptanzkriterien)
 - **DoD Stufe 1:** Eine Userstory ist seitens Entwicklung fertig umgesetzt
 - **DoD Stufe 2:** Eine Story ist technisch und funktional integriert
 - **DoD Stufe 3:** Eine Story ist als Bestandteil eines Releases freigegeben
 - **DoD Stufe 4:** Eine Story ist Produktiv
- Die Beschreibung der DoD Stufen sollte sich dabei an **existierenden Staging-Stufen und Verantwortungsübergängen** orientieren. Wichtig an dieser Stelle ist zu erwähnen, das CI/CD eine technisch orientierte Herangehensweise ist, die schnelles deployen als Ziel unterstützt.





1. Erläutere mit eigenen Worten die **Notwendigkeit** des **Testens** in Softwareprojekten!
2. Was besagt die **10er-Regel**?
3. Erkläre welcher **Fokus** bei den folgenden **Teststufen** gesetzt ist:
 1. **Unit Test**
 2. **Integrations- / System Test**
 3. **Akzeptanztest / Abnahmetest**
4. Was besagt das Prinzip der **Testpyramide**?
5. Nenne **5 Testarten**!
6. Erläutere das Prinzip „**Test Driven Development (TDD)**“!
7. Was ist eine **Definition of Done** und wieso kann es sinnvoll sein, mehrere **Stufen** davon zu haben?

TESTEN MIT PYTHON

python unittest

- The **unittest** testing framework was originally inspired by JUnit and has a similar flavor as major unit testing frameworks in other languages. It supports test automation, sharing of setup and shutdown code for tests, aggregation of tests into collections, and independence of the tests from the reporting framework.

Link: <https://docs.python.org/3/library/unittest.html>



pytest

- The **pytest** framework makes it easy to write small, readable tests, and can scale to support complex functional testing for applications and libraries.

Link: <https://docs.pytest.org/en/stable/>



Viele **weitere Tools**, bspw. unter <https://wiki.python.org/moin/PythonTestingToolsTaxonomy>



test fixture

- A test fixture represents the preparation needed to perform one or more tests, and any associated cleanup actions. This may involve, for example, creating temporary or proxy databases, directories, or starting a server process.

test case

- A test case is the individual unit of testing. It checks for a specific response to a particular set of inputs. unittest provides a base class, TestCase, which may be used to create new test cases.

test suite

- A test suite is a collection of test cases, test suites, or both. It is used to aggregate tests that should be executed together.

test runner

- A test runner is a component which orchestrates the execution of tests and provides the outcome to the user. The runner may use a graphical interface, a textual interface, or return a special value to indicate the results of executing the tests.

Quelle: <https://docs.python.org/3/library/unittest.html>

UNITTEST BEISPIEL



- The **unittest** module provides a rich set of tools for **constructing and running tests**. This section demonstrates that a small subset of the tools suffice to meet the needs of most users.
- Here is a short script to test **three string methods**:

```
import unittest

class TestStringMethods(unittest.TestCase):

    def test_upper(self):
        self.assertEqual('foo'.upper(), 'FOO')

    def test_isupper(self):
        self.assertTrue('FOO'.isupper())
        self.assertFalse('Foo'.isupper())

    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', 'world'])
        # check that s.split fails when the separator is not a string
        with self.assertRaises(TypeError):
            s.split(2)

if __name__ == '__main__':
    unittest.main()
```

Quelle: <https://docs.python.org/3/library/unittest.html#basic-example>

UNITTEST BEISPIEL



- The **test case** is created by subclassing `unittest.TestCase`. The three individual tests are defined with methods whose names start with the letters test. This naming convention informs the test runner about which methods represent tests.
- The crux of each test is a **call to `assertEqual()`** to check for an expected result; `assertTrue()` or `assertFalse()` to verify a condition; or `assertRaises()` to verify that a specific exception gets raised. These methods are used instead of the `assert` statement so the test runner can accumulate all test results and produce a report.
- The `setUp()` and `tearDown()` methods allow you to define instructions that will be executed before and after each test method.
- The final block shows a simple way to run the tests. **`unittest.main()`** provides a command-line interface to the test script. When run from the command line, the above script produces an output that looks like this:

```
...  
-----  
Ran 3 tests in 0.000s  
  
OK
```

Quelle: <https://docs.python.org/3/library/unittest.html#basic-example>

UNITTEST BEISPIEL



- Passing the -v option to your test script will instruct unittest.main() to enable a higher level of verbosity, and produce the following output:

```
test_isupper (__main__.TestStringMethods.test_isupper) ... ok
test_split (__main__.TestStringMethods.test_split) ... ok
test_upper (__main__.TestStringMethods.test_upper) ... ok
```

```
-----
Ran 3 tests in 0.001s
```

```
OK
```

- The above examples show the most commonly used unittest features which are sufficient to meet many everyday testing needs.

UNITTEST

COMMAND LINE INTERFACE



- The unittest module can be used from the command line to run tests from modules, classes or even individual test methods:

```
python -m unittest test_module1 test_module2
python -m unittest test_module.TestClass
python -m unittest test_module.TestClass.test_method
```

- You can pass in a list with any combination of module names, and fully qualified class or method names.
- Test modules can be specified by file path as well:

```
python -m unittest tests/test_something.py
```

- This allows you to use the shell filename completion to specify the test module. The file specified must still be importable as a module. The path is converted to a module name by removing the `‘.py’` and converting path separators into `‘.’`. If you want to execute a test file that isn’t importable as a module you should execute the file directly instead.

UNITTEST

COMMAND LINE INTERFACE



- You can run tests with more detail (higher verbosity) by passing in the -v flag:

```
python -m unittest -v test_module
```

- When executed without arguments Test Discovery is started:

```
python -m unittest
```

- For a list of all the command-line options:

```
python -m unittest -h
```

- **-b, --buffer** - The standard output and standard error streams are buffered during the test run. Output during a passing test is discarded. Output is echoed normally on test fail or error and is added to the failure messages.
- **-c, --catch** - Control-C during the test run waits for the current test to end and then reports all the results so far.
- **-f, --failfast** - Stop the test run on the first error or failure.
- **-k**, Only run test methods and classes that match the pattern or substring. This option may be used multiple times, in which case all test cases that match any of the given patterns are included.

Quelle: <https://docs.python.org/3/library/unittest.html#command-line-interface>

UNITTEST

TEST DISCOVERY



- Unittest supports simple **test discovery**. In order to be compatible with test discovery, all of the test files must be modules or packages importable from the top-level directory of the project (this means that their filenames must be valid identifiers).
- Test discovery is implemented in `TestLoader.discover()`, but can also be used from the command line. The basic command-line usage is:

```
cd project_directory  
python -m unittest discover
```

Note: As a shortcut, `python -m unittest` is the equivalent of `python -m unittest discover`. If you want to pass arguments to test discovery the `discover` sub-command must be used explicitly.

UNITTEST TEST DISCOVERY



- The discover sub-command has the following options:
- **-v, --verbose**
- Verbose output
- **-s, --start-directory directory**
- Directory to start discovery (. default)
- **-p, --pattern pattern**
- Pattern to match test files (test*.py default)
- **-t, --top-level-directory directory**
- Top level directory of project (defaults to start directory)
- The **-s, -p, and -t options** can be passed in as positional arguments in that order. The following two command lines are equivalent:

```
python -m unittest discover -s project_directory -p "*_test.py"  
python -m unittest discover project_directory "*_test.py"
```

- As well as being a path it is possible to pass a package name, for example myproject.subpackage.test, as the start directory. The package name you supply will then be imported and its location on the filesystem will be used as the start directory.

Quelle: <https://docs.python.org/3/library/unittest.html#test-discovery>

BEISPIEL

Unittest erstellen:

Testet einzelne Funktionen isoliert (z.B. clean_data)

Integrationstest erstellen

Testet den Ablauf mehrerer Komponenten zusammen (z.B. Daten laden, säubern, Modell trainieren, Modell bewerten).

Ausführen

- `python -m unittest discover -s tests`
- `python -m unittest tests/test_preprocessing.py`
- `python -m unittest tests/test_integration.py`

PROJEKTBAUM

```
– uc_app/
– |—— data/
– |   |—— audi.csv      # Rohdaten (z. B. aus Kaggle)
– |
– |—— src/
– |   |—— __init__.py    # Macht src zum Package
– |   |—— preprocessing.py # Daten laden & bereinigen
– |   |—— modeling.py    # Modell-Training & ggf. Vorhersage
– |
– |—— tests/
– |   |—— __init__.py
– |   |—— test_preprocessing.py # Unit-Tests für Preprocessing
– |   |—— test_modeling.py    # Unit-Tests für Modell-Training
– |   |—— test_integration.py # Integrationstest (Pipeline-Ende-zu-Ende)
– |
– |—— requirements.txt    # Python-Abhängigkeiten
– |—— README.md          # Projektbeschreibung
– |—— main.py            # (Optional) Hauptskript für Training oder Deployment
```


UNITTEST - PREPROCESSING

```
import unittest
import pandas as pd
from src.preprocessing import load_data, clean_data

class TestPreprocessing(unittest.TestCase):

    def test_clean_data_removes_nan(self):
        # Beispiel DataFrame mit fehlenden Werten
        data_path = "data/audi.csv"
        df = load_data(data_path)
        cleaned_df = clean_data(df)

        # Überprüfen ob keine NaNs mehr vorhanden
        self.assertFalse(cleaned_df.isnull().values.any(), "Es sind noch NaN-Werte enthalten.")

if __name__ == '__main__':
    unittest.main()
```

INTEGRATIONSTEST

```
import unittest
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score

from src.preprocessing import load_data, clean_data
from src.modeling import train_model

class TestIntegration(unittest.TestCase):

    def test_full_pipeline(self):
        data_path = "data/audi.csv"

        # Daten Laden und bereinigen
        df = load_data(data_path)
        df_clean = clean_data(df)

        # Features und Zielvariable
        X = df_clean[['year', 'mileage']]
        y = df_clean['price']

        # Train/Test-Split
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=0.2, random_state=42
        )

        # Modell trainieren
        model = train_model(X_train, y_train)

        # Prüfen, ob Modell trainiert wurde
        self.assertTrue(hasattr(model, 'coef_'))
        self.assertEqual(len(model.coef_), X.shape[1])

        # Vorhersagen auf Testdaten
        predictions = model.predict(X_test)

        # Fehlerbewertung
        mse = mean_squared_error(y_test, predictions)
        self.assertGreaterEqual(mse, 0.0, "MSE darf nicht negativ sein")

        # R² auf Testdaten
        r2 = r2_score(y_test, predictions)
        self.assertGreater(r2, 0.5, "R² zu niedrig - Modell passt nicht gut")

if __name__ == '__main__':
    unittest.main()
```

```
from src.preprocessing import load_data, clean_data
from src.modeling import train_model
#from src.eda import exploratory_data_analysis

# Pfad zu den Daten
data_path = "data/audi.csv"

# Daten laden und aufbereiten
df = load_data(data_path)
df = clean_data(df)

# Features und target
X = df[["year", "mileage"]]
y = df["price"]

model = train_model(X, y)
print("Modell erfolgreich trainiert!")
```

Testing

25 Minuten Zeit



Aufgabe:

Erstelle Tests für deine Gebrauchtwagen-Applikation.

1. Erstelle ein neues Projektrepository mit den folgenden Ordnern: **data**, **src**, **tests** und verschiebe die **csv-datei** nach **data**
2. Teile das letzte notebook in mehrere **python files** auf
 - Wähle eine sinnvolle Trennung, bspw. nach den CRISP-DM-Phasen in **Prepration/Preprocessing**, **Modeling**, **Evaluation**, **Deployment**
 - Erstelle eine **main** Datei, die als Startpunkt alle Teile zusammenführt
3. Erstelle unter **tests** jeweils **einen unittest pro python file** und mindestens **einen integrationstest** (ob mittels python unittest oder pytest bleibt dir überlassen).



Lernziele



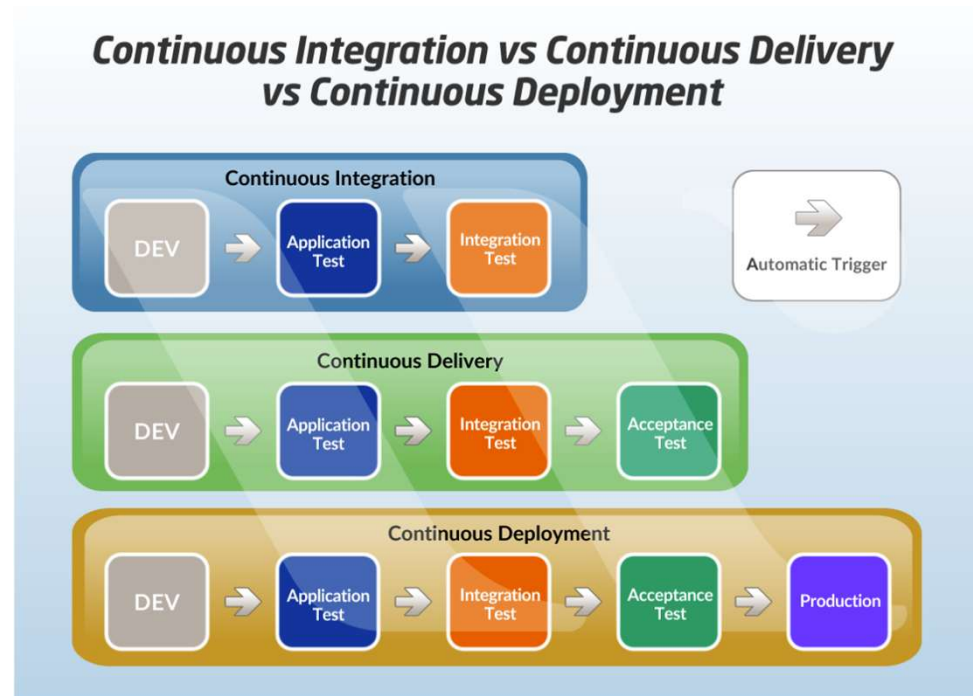
Nach der Bearbeitung dieser Lektion werdet ihr wissen, ...

- Warum das **Testen** von **Applikationen** wichtig ist
 - Was man unter einem **Unit-Test** versteht
 - Was man unter einem **Integrations-Test** versteht
 - Was man unter **“Test-Driven Development” (TDD)** versteht
 - Besonderheiten beim **Testen** in **agilen Projekten**
- Wie man ein **lokales Modell** in eine verteilte **Produktionsumgebung** bringt
 - Was man unter **„Continuous Integration (CI)“** / „Kontinuierliche Integration“ versteht
 - Was man unter **„Continuous Delivery (CD)“** / „Kontinuierliche Auslieferung“ versteht

CI/CD

CONTINUOUS INTEGRATION / CONTINUOUS DELIVERY

- **CI/CD** steht für **Continuous Integration** und **Continuous Delivery** (bzw. **Deployment**) und beschreibt eine moderne Praxis zur **automatisierten Softwareentwicklung, -test und -bereitstellung**.



Quelle: <https://www.whizlabs.com/blog/continuous-integration-vs-continuous-delivery-vs-continuous-deployment/>

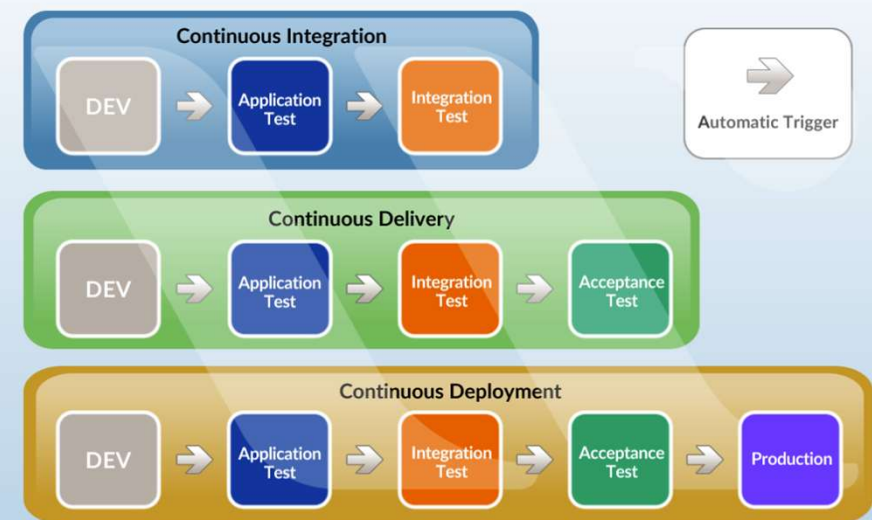
Quelle: <https://www.whizlabs.com/blog/continuous-integration-vs-continuous-delivery-vs-continuous-deployment/>

CONTINUOUS INTEGRATION

Bei “**Continuous Integration (CI)**” werden die Änderungen von Entwicklern so häufig wie möglich in die gemeinsame Codebasis gemergt.

- Änderungen werden durch (automatisierte) Tests validiert
- Durch regelmäßige Integration und Tests wird ein großer Teil Integrationsarbeit für einen geplanten “Release Day” vorweggenommen
- Die frühzeitige Ausführung von Tests soll dabei helfen aufkommende Fehler schnell zu finden und relativ kostengünstig zu beheben (vergleiche 10er Regel)
- Die Automatisierung von Tests hilft dabei, den Prozess zu beschleunigen

Continuous Integration vs Continuous Delivery vs Continuous Deployment



Quellen: <https://www.atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment>

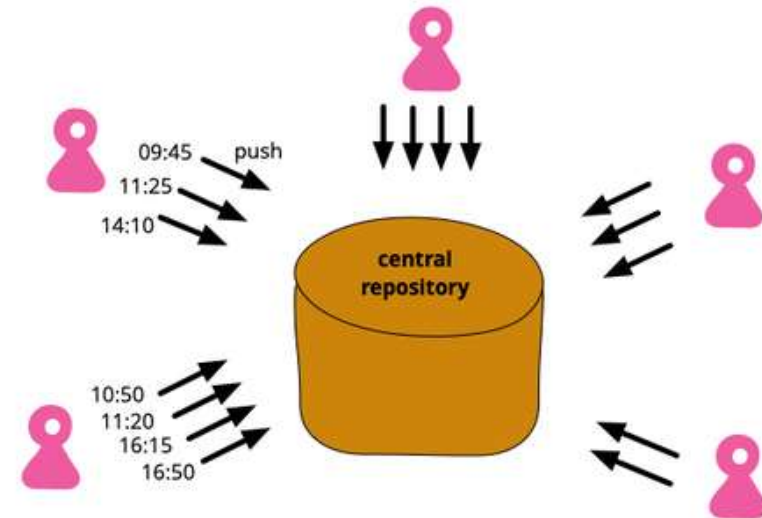
CONTINUOUS INTEGRATION

„**Continuous Integration** is a **software development practice** where each member of a team **merges** their **changes** into a **codebase** together with their colleagues changes **at least daily**.

Each of these integrations is **verified** by an **automated build** (including test) to **detect integration errors** as quickly as possible.

Teams find that this approach reduces the **risk of delivery delays**, reduces the **effort of integration**, and enables practices that foster a **healthy codebase** for rapid enhancement with new features.”

(Martin Fowler, 2024)



Quelle: Martin Fowler,
<https://martinfowler.com/articles/continuousIntegration.html>

CONTINUOUS INTEGRATION

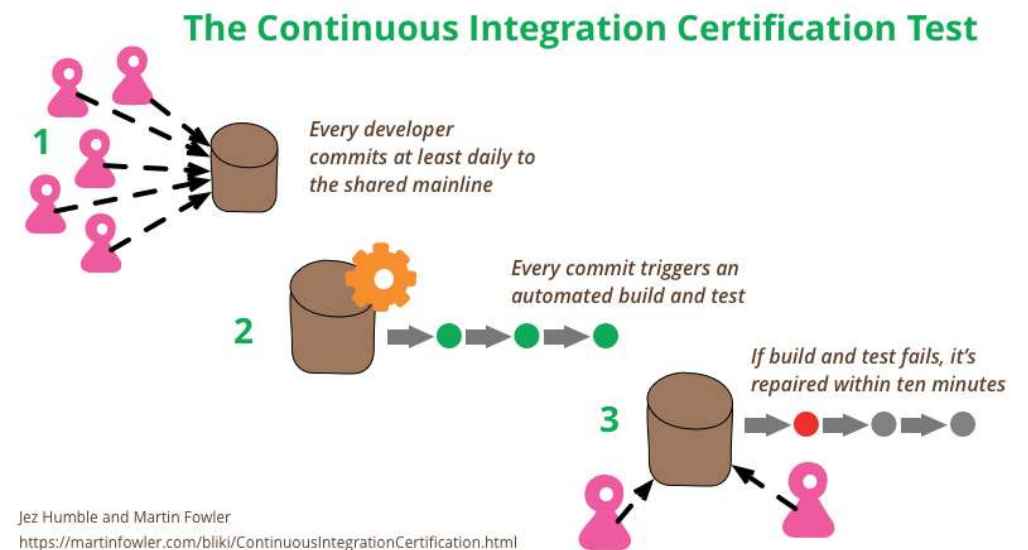
Good Practices of Continuous Integration:

- **Put everything in a version controlled mainline**
- **Automate the Build**
- **Make the Build Self-Testing**
- **Everyone Pushes Commits To the Mainline Every Day**
- **Every Push to Mainline Should Trigger a Build**
- **Fix Broken Builds Immediately**
- **Keep the Build Fast**
- **Hide Work-in-Progress**
- **Test in a Clone of the Production Environment**
- **Everyone can see what's happening**
- **Automate Deployment**

(Martin Fowler, 2024)

Quellen: <https://martinfowler.com/articles/continuousIntegration.html>

24.06.2025 433 DS_HAN_DSSE042401_SoSe25; Kurs: Data Science Software Engineering, Dozent: Dr. Christopher Kiefer



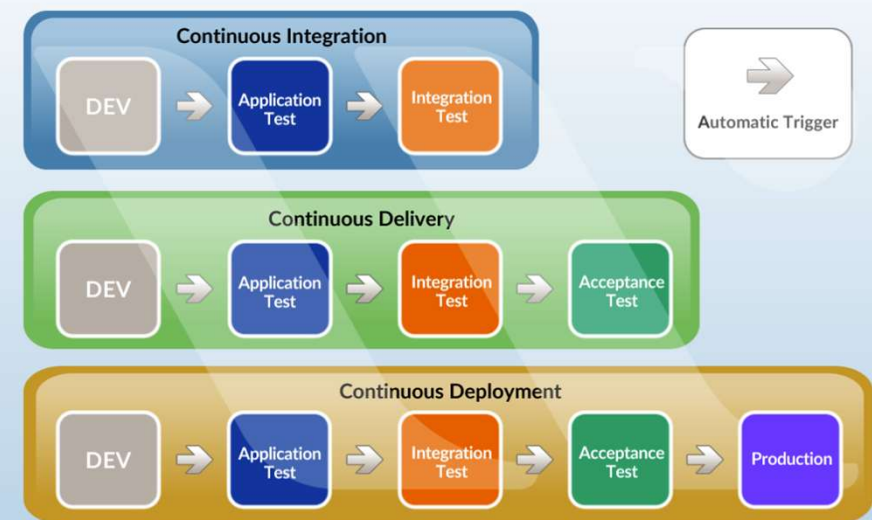
Quelle: Martin Fowler,
<https://martinfowler.com/articles/continuousIntegration.html>

CONTINUOUS DELIVERY

“**Continuous delivery (CD)** ist eine Erweiterung von Continuous Integration (CI).

- Code-Änderungen werden automatisch an eine Test- und /oder Vor-Produktionsumgebung ausgeliefert.
- der automatische Integrations- und Testprozess wird dabei um einen automatischen Auslieferungsprozess erweitert
- Eine Auslieferung an die Produktionsumgebung kann somit häufiger und flexibler gestartet werden (wöchentlich, täglich, über Nacht, je nach Anforderung)
- Um die Vorteile von Continuous Delivery zu nutzen, sollten dabei fertige Inkremente so früh wie möglich ausgeliefert werden.
- Kleine Produktinkremente erleichtern auch die Fehlerbehebung bei aufkommenden Problemen

Continuous Integration vs Continuous Delivery vs Continuous Deployment



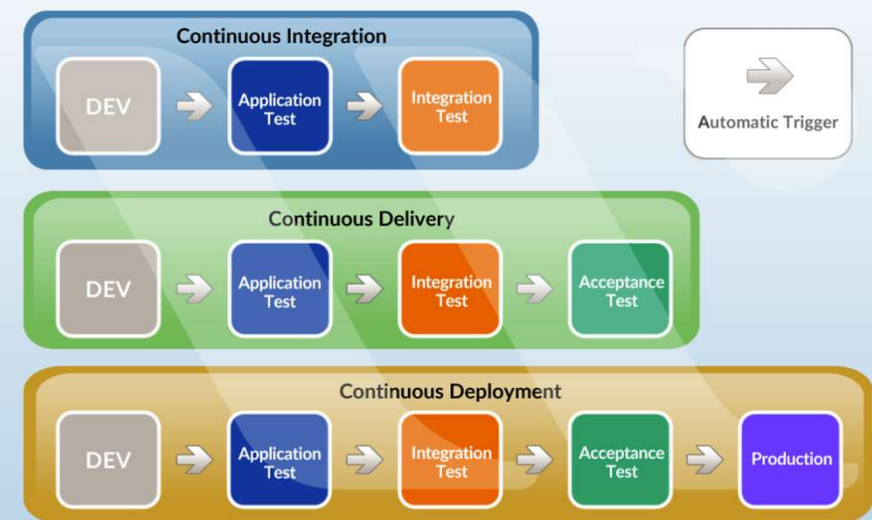
Quellen: <https://www.atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment>

CONTINUOUS DEPLOYMENT

“**Continuous deployment**” geht noch einen Schritt weiter als continuous delivery.

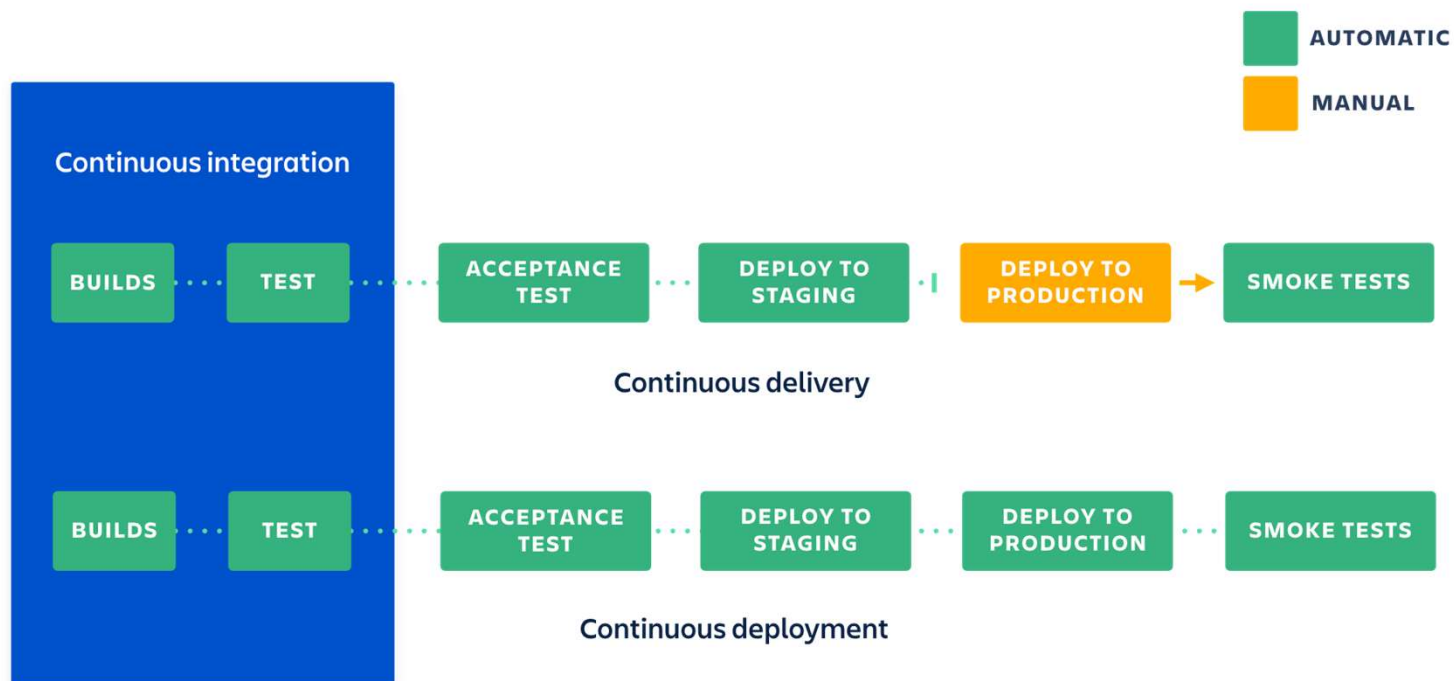
- Alle Änderung, die bereits die Stages bis zur Produktionsumgebung durchlaufen haben, werden automatisch als neues Release an den Kunden ausgeliefert.
- Dafür sind keine manuellen Aktivitäten mehr notwendig
- Lediglich fehlgeschlagene Tests führen dazu, dass eine Änderung nicht auf die Produktionsumgebung ausgerollt wird.
- Daruch ist schnelles und direktes Feedback vom Kunden möglich
- Entwickler können sich auf die Softwareentwicklung konzentrieren und müssen keinen geplanten “Release Day” mehr vorbereiten
- Änderungen sind unmittelbar im Produktionssystem nutzbar
- Jedoch entsteht Risiko unbemerkte Fehler ebenfalls schnell auszuliefern

Continuous Integration vs Continuous Delivery vs Continuous Deployment



Quellen: <https://www.atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment>

VERGLEICH CONTINUOUS INTEGRATION, CONTINUOUS DELIVERY UND CONTINUOUS DEPLOYMENT



Quelle: <https://www.atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment>

Quelle: <https://www.atlassian.com/continuous-delivery/principles/continuous-integration-vs-delivery-vs-deployment>

ÜBERSICHT

Merkmal	Continuous Integration (CI)	Continuous Delivery (CD)	Continuous Deployment
Scope	Regelmäßiges Integrieren von Code in ein gemeinsames Repo	Automatisierte Vorbereitung zur Auslieferung nach jedem Commit	Automatische Auslieferung in Produktion nach jedem Commit
Ziel	Frühe Fehlererkennung und Integration	Jederzeit auslieferbare Software	Vollständig automatisierter Release-Prozess
Automatisierung	Build, Unit-Tests	Build, Tests, Deployment bis Staging	Build, Tests, Deployment bis Produktion
Deployment in Produktion	Manuell	Manuell oder auf Knopfdruck	Automatisch nach jedem erfolgreichen Build/Test
Häufige Tools	Jenkins, GitHub Actions, GitLab CI	Spinnaker, ArgoCD, GitLab CD	wie CD, zusätzlich mit Canary Release, Blue/Green
Herausforderungen	Testabdeckung, Merge-Konflikte	Qualitätssicherung, Teststabilität	Vertrauen in Tests, Feature Flags, Monitoring

EINIGE TOOLS FÜR CONTINUOUS INTEGRATION (CI)

Bitbucket Pipelines	Jenkins	AWS CodePipeline	Circle CI	Azure Pipelines	GitLab	Atlassian Bamboo
• Easy setup and configuration • Unified Bitbucket experience • Cloud by 3rd party	• On-premise • Open source • Robust addon / plugin ecosystem	• Fully cloud • Integrated with Amazon Web services • Custom plugin support • Robust access control	• Notification triggers from CI events • Performance optimized for quick builds • Easy debugging through SSH and local builds • Analytics to measure build performance	• Azure platform integration • Windows platform support • Container support • Github integration	• On-prem or cloud hosting • Continuous security testing • Easy to learn UX	• Best integration with Atlassian product suite • An extensive market place of add-ons and plugins • Container support with Docker agents • Trigger API for IFTTT functionality

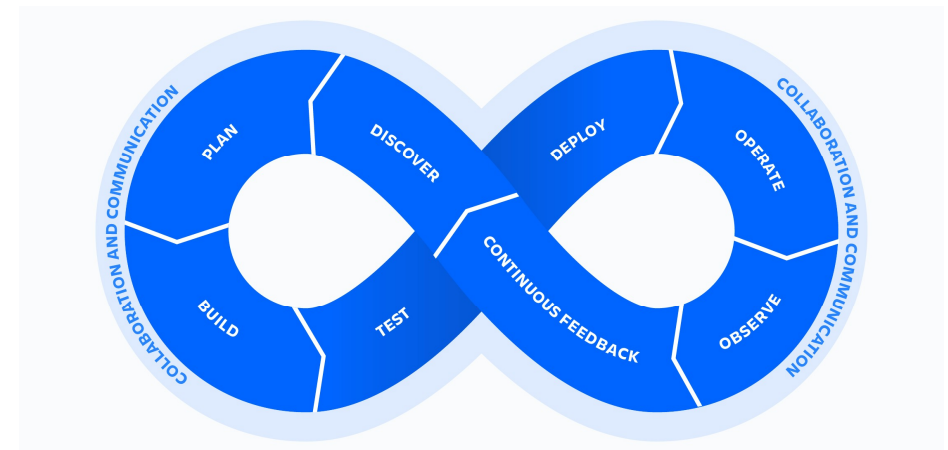
Quelle: <https://www.atlassian.com/continuous-delivery/continuous-integration/tools>

DEVOPS

DevOps ist ein Kunstwort, das sich aus den Begriffen "**Development**" (**Entwicklung**) und "**Operations**" (**Betrieb**) zusammensetzt. Es bezeichnet eine **Kultur und Praxis** in der Softwareentwicklung, bei der Entwicklungs- und IT-Betriebsteams **eng zusammenarbeiten**, um Software schneller, effizienter und zuverlässiger bereitzustellen. Ziel ist es, den gesamten Lebenszyklus von Software – von der Entwicklung über das Testen bis hin zum Betrieb – durch Automatisierung, kontinuierliche Integration, kontinuierliche Lieferung (CI/CD) und Monitoring zu optimieren.

Kernprinzipien von DevOps:

- **Kollaboration:** Enge Zusammenarbeit zwischen Entwicklern und Betriebsverantwortlichen.
- **Automatisierung:** Automatisierte Tests, Builds, Deployments und Infrastrukturmanagement.
- **Kontinuierliche Integration und Lieferung (CI/CD):** Häufiges, zuverlässiges Ausliefern neuer Softwareversionen.
- **Monitoring & Feedback:** Kontinuierliches Überwachen von Anwendungen zur schnellen Fehlererkennung und Verbesserung.



Quelle: <https://www.atlassian.com/devops>

Quelle: <https://www.atlassian.com/devops>

Figure 1: Magic Quadrant for DevOps Platforms



Gartner

Quelle: <https://www.atlassian.com/gartner/magic-quadrant-devops>

Quelle: <https://www.atlassian.com/gartner/magic-quadrant-devops>

Integration / Delivery

Übungsfragen



1. Grenze die folgenden drei Begriffe gegeneinander ab:
 - **Continuous Integration (CI)**
 - **Continuous Delivery (CD)**
 - **Continuous Deployment**
2. Warum ist es insbesondere für **agile Projekte** wichtig, die Konzepte **Continuous Integration (CI)** und **Continuous Delivery (CD)** zu berücksichtigen?
3. Erläutere den Begriff **DEVOPS** und wie er sich zusammensetzt!



GitHub ist eine cloubasierte Plattform, auf der du deinen Code **speichern, teilen** und **gemeinsam mit anderen daran arbeiten** kannst.

Wenn du deinen Code in einem sogenannten „**Repository**“ auf GitHub speicherst, ermöglicht dir das:

- Deine Arbeit zu präsentieren oder mit anderen zu teilen.
- Änderungen an deinem Code im Laufe der Zeit nachzuvollziehen und zu verwalten.
- Anderen die Möglichkeit zu geben, deinen Code zu überprüfen und Verbesserungsvorschläge zu machen.
- Gemeinsam an einem Projekt zu arbeiten – ohne Sorge, dass deine Änderungen die Arbeit anderer beeinträchtigen, bevor du sie wirklich einbinden willst.

Diese **kollaborative Zusammenarbeit**, eine der zentralen Funktionen von GitHub, wird durch die Open-Source-Software **Git** ermöglicht, auf der GitHub basiert.

GIT – EIN VERSIONSKONTROLLSYSTEM

Git ist ein Versionskontrollsystem, das Änderungen an Dateien intelligent nachverfolgt. Git ist besonders nützlich, wenn du **gemeinsam mit mehreren Personen gleichzeitig an denselben Dateien arbeitest**.

Typischerweise sieht ein **Git-basierter Workflow** so aus:

- Du **erstellst einen Branch (Zweig)** aus der Hauptversion der Dateien, an der du und deine Teammitglieder arbeiten.
- Du **bearbeitest die Dateien unabhängig und sicher** in deinem eigenen, persönlichen Branch.
- Git **führt deine spezifischen Änderungen intelligent mit der Hauptversion der Dateien zusammen**, sodass deine Anpassungen die Änderungen anderer nicht stören.
- Git **verfolgt alle Änderungen – deine und die der anderen –**, sodass ihr immer auf dem neuesten Stand des Projekts arbeitet.

WIE ARBEITEN GIT UND GITHUB ZUSAMMEN?

Wenn du Dateien auf GitHub hochlädst, speicherst du sie in einem sogenannten „**Git-Repository**“. Das bedeutet, dass Git automatisch beginnt, deine Änderungen (sogenannte „**Commits**“) an den Dateien zu verfolgen und zu verwalten, sobald du etwas in GitHub änderst.

Viele git-bezogene Aktionen kannst du direkt im Browser auf GitHub durchführen, zum Beispiel:

- **ein Git-Repository erstellen,**
- **Branches (Zweige) anlegen,**
- **Dateien hochladen und bearbeiten.**

Die meisten Entwickler*innen arbeiten jedoch **lokal auf ihrem eigenen Computer** an den Dateien und synchronisieren ihre lokalen Änderungen regelmäßig mit dem zentralen (sogenannten „**Remote**“-)**Repository** auf GitHub. Dafür gibt es verschiedene Tools, wie zum Beispiel **GitHub Desktop**.

Sobald du mit anderen gemeinsam am selben Repository arbeitest, wirst du ständig:

- alle aktuellen Änderungen deiner Teammitglieder vom Remote-Repository auf GitHub **herunterladen (pull)**,
- deine eigenen Änderungen zurück an das Remote-Repository auf GitHub **hochladen (push)**.

Git sorgt dann dafür, dass diese Änderungen intelligent zusammengeführt (gemerged) werden. **GitHub** unterstützt dich dabei mit Funktionen wie „**Pull Requests**“, um diesen Ablauf besser zu verwalten.

Quelle: <https://docs.github.com/en/get-started/start-your-journey/about-github-and-git>

Signing up for a new personal account

- 1 Navigate to <https://github.com/>.
- 2 Click Sign up.
- 3 Follow the prompts to create your personal account.

Public profile

Name

The Octocat

Your name may appear around GitHub where you contribute or are mentioned. You can remove it at any time.

Public email

X Remove

You can manage verified email addresses in your [email settings](#).

Bio

Profile picture



Erste Schritte:

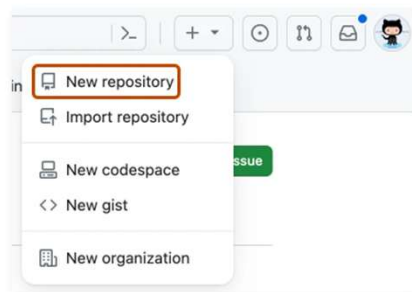
1. Account erstellen
2. Profil einrichten
3. Repository erstellen
4. Synchronisieren
5. Inspiration finden
6. Projekte erstellen

<https://docs.github.com/en/get-started>

GITHUB

REPOSITORY ERSTELLEN

- 1 In the upper-right corner of any page, select +, then click **New repository**.



- 2 In the "Repository name" box, type `hello-world`.
- 3 In the "Description" box, type a short description. For example, type "This repository is for practicing the GitHub Flow."
- 4 Select whether your repository will be **Public** or **Private**.
- 5 Select **Add a README file**.
- 6 Click **Create repository**.

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * / Repository name *
pytest
✔ pytest is available.

Great repository names are short and memorable. Need inspiration? How about **turbo-telegram** ?

Description (optional)

☐ Public
Anyone on the internet can see this repository. You choose who can commit.
☒ Private
You choose who can see and commit to this repository.

Initialize this repository with:

☒ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: None

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

This will set `main` as the default branch. Change the default name in your [settings](#).

ⓘ You are creating a private repository in your personal account.

Create repository

Quelle: <https://docs.github.com/en/get-started/start-your-journey/hello-world>

GITHUB

BRANCHING – ARBEITEN MIT VERSIONSZWEIGEN

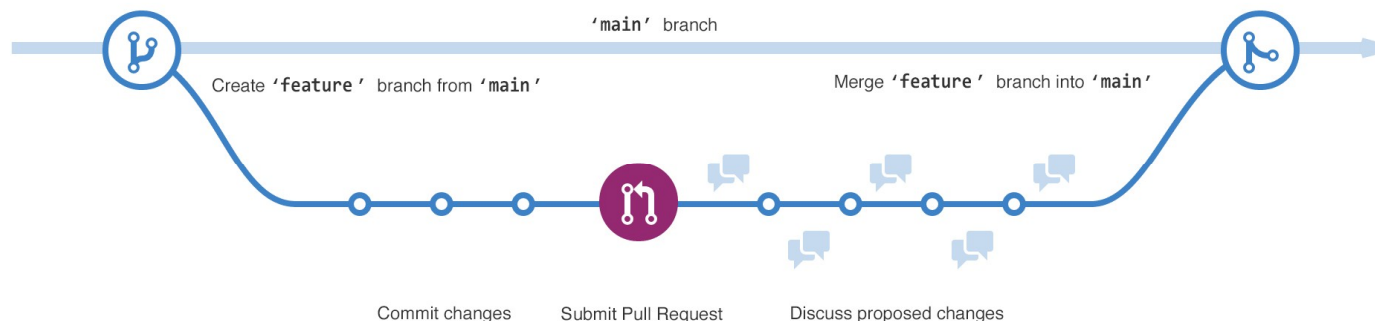
Branching ermöglicht es dir, mehrere Versionen eines Repositories gleichzeitig zu verwalten.

Standardmäßig enthält jedes Repository einen Branch namens **main**, der als die maßgebliche Hauptversion gilt.

Du kannst weitere Branches von main aus erstellen.

Branching ist besonders nützlich, wenn du neue Funktionen zu einem Projekt hinzufügen möchtest, ohne den Hauptcode direkt zu verändern. Die Arbeit auf anderen Branches erscheint erst dann in main, wenn du sie zusammenführst (**merge**). Mit Branches kannst du also experimentieren und Änderungen testen, bevor du sie in die Hauptversion übernimmst.

Wenn du einen neuen Branch von **main** erstellst, erzeugst du damit eine **Kopie** (einen **Snapshot**) des Zustands von main zu diesem Zeitpunkt. Wenn jemand währenddessen Änderungen an main vorgenommen hat, kannst du diese später in deinen Branch übernehmen (**pullen**).



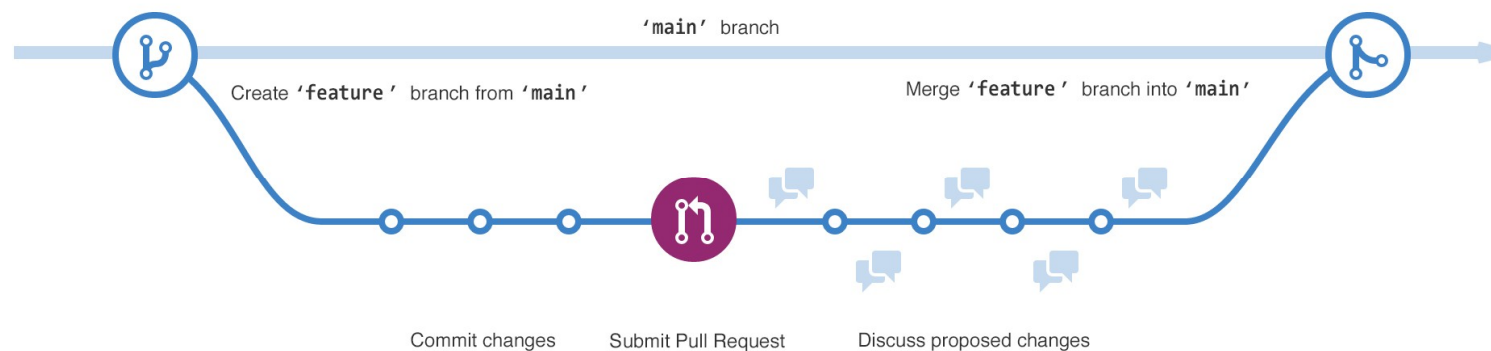
Quelle: <https://docs.github.com/en/get-started/start-your-journey/hello-world>

GITHUB

BRANCHING – ARBEITEN MIT VERSIONSZWEIGEN

Das Diagramm zeigt

- den **Haupt-Branch (main)**,
- einen **neuen Branch namens feature**,
- den Weg, den feature durchläuft:
 - **Änderungen committen**,
 - **Pull Request einreichen**,
 - **Vorgeschlagene Änderungen diskutieren**,
 - bevor der Branch schließlich in main **zusammengeführt (gemerged)** wird.



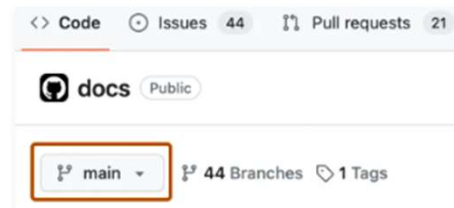
Quelle: <https://docs.github.com/en/get-started/start-your-journey/hello-world>

GITHUB

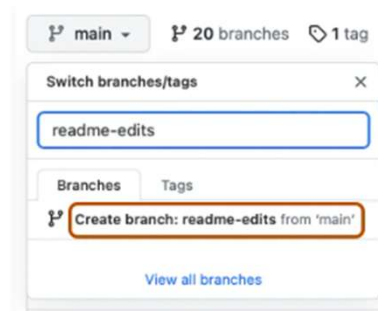
BRANCHING – BRANCHE ERSTELLEN

Creating a branch [↗](#)

- 1 Click the **Code** tab of your `hello-world` repository.
- 2 Above the file list, click the dropdown menu that says **main**.



- 3 Type a branch name, `readme-edits`, into the text box.
- 4 Click **Create branch: readme-edits from main**.



Now you have two branches, `main` and `readme-edits`. Right now, they look exactly the same. Next you'll add changes to the new `readme-edits` branch.

Quelle: <https://docs.github.com/en/get-started/start-your-journey/hello-world>

GITHUB

BRANCHING – CHANGE AND COMMIT

Als du im vorherigen Schritt einen neuen Branch erstellt hast, hat dich GitHub zur **Code-Seite deines neuen Branches** weitergeleitet – dieser ist eine **Kopie des main-Branches**.

Du kannst nun **Änderungen an den Dateien in deinem Repository vornehmen und speichern**. Auf GitHub werden gespeicherte Änderungen „**Commits**“ genannt.

Jeder Commit hat eine sogenannte **Commit-Nachricht** – das ist eine kurze Beschreibung, **warum eine bestimmte Änderung vorgenommen wurde**.

Commit-Nachrichten dokumentieren den Verlauf deiner Änderungen, sodass **andere Mitwirkende nachvollziehen können, was du geändert hast und warum**.

- 1 Under the `readme-edits` branch you created, click the `README.md` file.
- 2 To edit the file, click .
- 3 In the editor, write a bit about yourself.
- 4 Click **Commit changes**.
- 5 In the “Commit changes” box, write a commit message that describes your changes.
- 6 Click **Commit changes**.

These changes will be made only to the README file on your `readme-edits` branch, so now this branch contains content that's different from `main`.

GITHUB

BRANCHING – PULL REQUEST

- Da du Änderungen in einem Branch abseits von main vorgenommen hast, kannst du jetzt einen **Pull Request** öffnen.
- **Pull Requests** sind das Herzstück der Zusammenarbeit auf GitHub. Wenn du einen Pull Request erstellst, **schlägst du deine Änderungen vor** und **bittest jemanden, diese zu überprüfen und in seinen Branch zu übernehmen**.
- Pull Requests zeigen sogenannte **Diffs** – also die **Unterschiede** zwischen den Inhalten beider Branches. Änderungen, Hinzufügungen und Löschungen werden **farblich hervorgehoben** dargestellt.
- Sobald du einen Commit gemacht hast, kannst du einen Pull Request öffnen und **eine Diskussion starten**, selbst wenn dein Code noch nicht vollständig fertig ist.
- Sobald du **mit anderen zusammenarbeitest**, ist jetzt der richtige Zeitpunkt, um sie **um eine Überprüfung (Review)** zu bitten.
- Dadurch haben deine Teammitglieder die Möglichkeit, **Kommentare abzugeben oder Änderungsvorschläge** zu deinem Pull Request zu machen – **bevor du die Änderungen in den main-Branch zusammenführst**
- .

Quelle: <https://docs.github.com/en/get-started/start-your-journey/hello-world>

- 1 Click the **Pull requests** tab of your `hello-world` repository.
- 2 Click **New pull request**.
- 3 In the **Example Comparisons** box, select the branch you made, `readme-edits`, to compare with `main` (the original).
- 4 Look over your changes in the diffs on the Compare page, make sure they're what you want to submit.



- 5 Click **Create pull request**.
- 6 Give your pull request a title and write a brief description of your changes. You can include emojis and drag and drop images and gifs.
- 7 Click **Create pull request**.

GITHUB

BRANCHING – MERGE

- In diesem letzten Schritt wirst du deinen Branch **mit dem main-Branch zusammenführen (mergen)**.
- Nachdem du den Pull Request gemergt hast, werden die Änderungen aus dem Branch **in den main-Branch übernommen**.
- Manchmal kann ein Pull Request **Änderungen enthalten, die mit dem bestehenden Code im main-Branch in Konflikt stehen**. In solchen Fällen informiert dich GitHub über diese **Konflikte** und verhindert das Mergen, **bis die Konflikte behoben sind**.
- Du kannst entweder selbst einen **Commit erstellen, der die Konflikte löst**, oder die **Kommentarfunktion im Pull Request nutzen**, um die Konflikte mit deinem Team zu besprechen.

- 1 At the bottom of the pull request, click **Merge pull request** to merge the changes into `main`.
- 2 Click **Confirm merge**. You will receive a message that the request was successfully merged and the request was closed.
- 3 Click **Delete branch**. Now that your pull request is merged and your changes are on `main`, you can safely delete the `readme-edits` branch. If you want to make more changes to your project, you can always create a new branch and repeat this process.
- 4 Click back to the **Code** tab of your `hello-world` repository to see your published changes on `main`.

GIT KOMMANDOS

- git **init** [project-name]
Legt ein neues lokales Repository mit dem angegebenen Namen an
- git **add** [file] / git add .
Indiziert den derzeitigen Stand der Datei / alle Dateien im aktuellen Verzeichnis und darunter für die Versionierung
- git **commit** -m"[descriptive message],,
Nimmt alle derzeit indizierten Dateien permanent in die Versionshistorie auf
- git **branch**
Listet alle lokalen Branches im aktuellen Repository auf
- git **branch** [branch-name]
Erzeugt einen neuen Branch
- git **fetch** [remote]
Lädt die gesamte Historie eines externen Repositories herunter
- git **merge** [remote]/[branch]
Integriert den externen Branch in den aktuell lokal ausgecheckten Branch
- git **push** [remote] [branch]
Pusht alle Commits auf dem lokalen Branch zu GitHub

Git Cheat Sheet: <https://training.github.com/downloads/de/github-git-cheat-sheet/>

Quelle: <https://training.github.com/downloads/de/github-git-cheat-sheet/>

GITHUB INSPIRATION FINDEN

GitHub.com ist die Heimat von Millionen Open-Source-Softwareprojekten, die du **kopieren**, **anpassen** und **für deine eigenen Zwecke nutzen** kannst.

Es gibt verschiedene Möglichkeiten, wie du eine Kopie der Dateien eines GitHub-Repositories erhalten kannst:

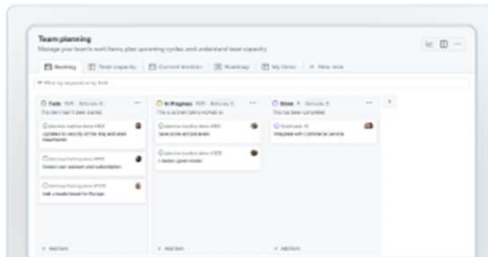
- Du kannst einen **Schnappschuss (Snapshot)** der Repository-Dateien als **ZIP-Datei herunterladen** und auf deinem lokalen Computer speichern.
- Du kannst das Repository mit **Git klonen** und eine vollständige Kopie lokal anlegen.
- Du kannst ein Repository **forken**, also eine **neue, eigene Version auf GitHub erstellen**.

Jede dieser Methoden hat ihren eigenen Anwendungsfall

GITHUB

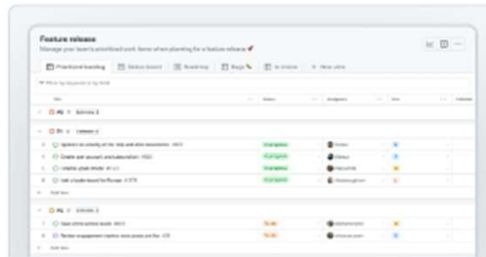
PROJEKT ERSTELLEN - TEMPLATES

Featured



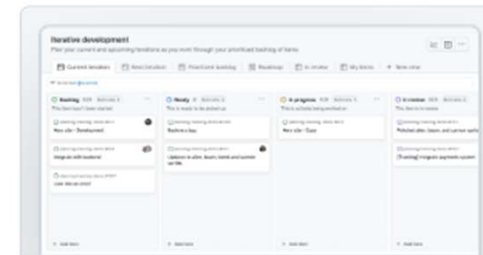
Team planning • GitHub

Manage your team's work items, plan upcoming cycles, and understand team capacity



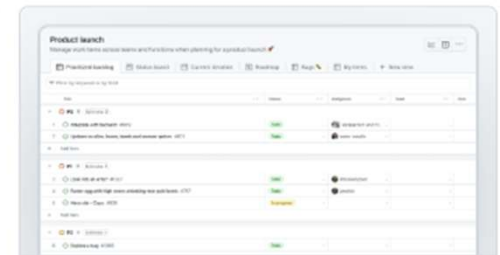
Feature release • GitHub

Manage your team's prioritized work items when planning for a feature release 🚀



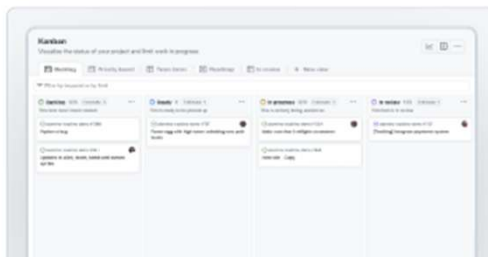
Iterative development • GitHub

Plan your current and upcoming iterations as you work through your prioritized backlog of items



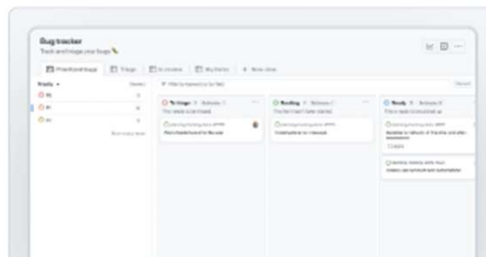
Product launch • GitHub

Manage work items across teams and functions when planning for a product launch 🚀



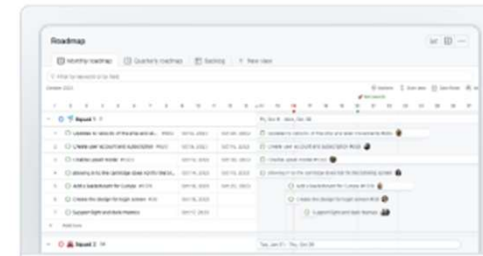
Kanban • GitHub

Visualize the status of your project and limit work in progress



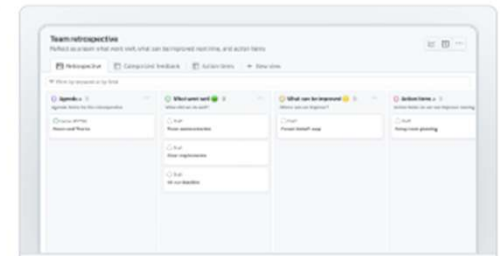
Bug tracker • GitHub

Track and triage your bugs 🐛



Roadmap • GitHub

Manage your team's long term plans as you plan out your roadmap



Team retrospective • GitHub

Reflect as a team what went well, what can be improved next time, and action items

GITHUB DESKTOP

Welcome to GitHub Desktop

Use this tutorial to get comfortable with Git, GitHub, and GitHub Desktop.



Git is the version control system.



GitHub is where you store your code and collaborate with others.



GitHub Desktop helps you work with GitHub locally.

Get started



- ✓ Install a text editor ✓
- 2 Create a branch ✓
- 3 Edit a file ✓
- 4 Make a commit ✓
- 5 Publish to GitHub ✓
- 6 Open a pull request ✓

Quelle: <https://git-scm.com/downloads>

GITHUB DESKTOP

You're done!

You've learned the basics on how to use GitHub Desktop. Here are some suggestions for what to do next.



Explore projects on GitHub

Contribute to a project that interests you

Open in browser



Create a new repository

Get started on a brand new project

Create repository



Add a local repository

Work on an existing project in GitHub Desktop

Add repository



Return to in progress tutorial



Clone a repository from the Internet...



Create a New Repository on your local drive...



Add an Existing Repository from your local drive...



Aufgabe:

Gemeinsame Arbeit an der Gebrauchtwagen-Applikation.

1. Findet euch in euren Zweier-Teams zusammen und erstellt ein **neues Git-Repository** und eine passende **README.md** bei Github
2. **Synchronisiert** die erstellte **Projektstruktur** aus der letzten Übung mit eurem neuen **Git-Repository** (data, src, tests, main)
3. Erstellt eine **Requirements.txt file** und pusht es über einen neuen Branch via commit changes in das Repository
4. Öffnet einen Pull-Request
5. Merged den Pull-Request mit dem main branch



Data Science Software Engineering

Teams

o3

1. Rudi
2. Krzysztof

mercedes

Hagenberg

1. Patryk
2. Tobias

ford

Housing Hackers

1. Manuel
2. Tijana

hyundai



kodo no timu

1. Sascha
2. Jeremy

toyota

CodeJetters

1. Vincent
2. Cynthia

bmw



GITHUB ACTIONS

GitHub Actions ist eine Plattform für **Continuous Integration und Continuous Delivery (CI/CD)**, mit der du deine **Build-, Test- und Deployment-Pipeline automatisieren** kannst.

Du kannst sogenannte Workflows erstellen, die beispielsweise:

- **bei jedem Pull Request** automatisch das Projekt **bauen und testen**,
- oder **nach dem Mergen** automatisch den Code **in die Produktionsumgebung deployen**.

GitHub Actions kann mehr als DevOps:

Du kannst Workflows auch ausführen, **wenn andere Ereignisse im Repository stattfinden**.

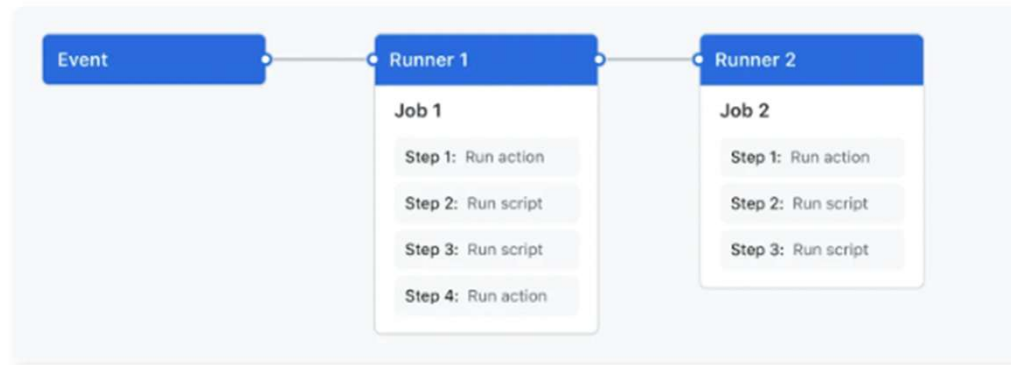
Zum Beispiel kannst du einen Workflow einrichten, der **automatisch passende Labels vergibt**, sobald jemand **ein neues Issue** in deinem Repository erstellt.

Quelle: <https://docs.github.com/en/actions/about-github-actions/understanding-github-actions>

GITHUB ACTIONS KOMPONENTEN

GitHub Action Komponenten

1. Workflows
2. Events
3. Jobs
4. Actions
5. Runners



Quelle: <https://docs.github.com/en/actions/about-github-actions/understanding-github-actions>

GITHUB ACTIONS KOMPONENTEN

GitHub Action Komponenten

1. Workflows

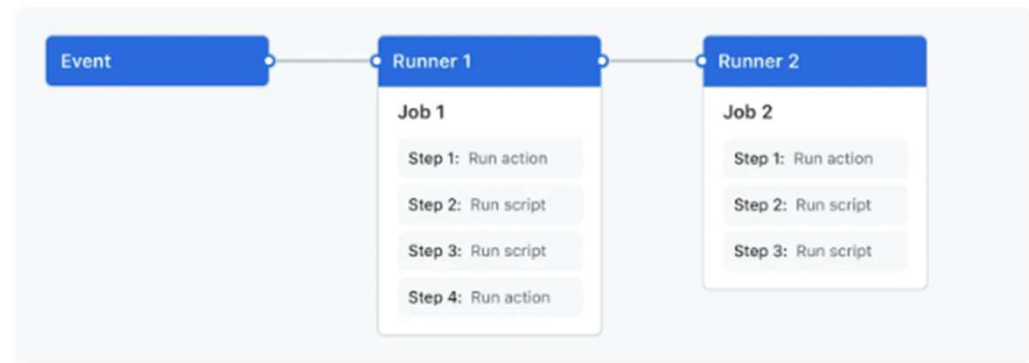
Ein Workflow ist ein **automatisierter Prozess**, der aus einem oder mehreren Jobs besteht.

Workflows werden als **YAML-Dateien** im Ordner **.github/workflows** gespeichert.

Sie können durch **Ereignisse**, **manuell** oder **zeitgesteuert** ausgelöst werden.

Ein Repository kann mehrere Workflows enthalten, z. B. zum:

- Testen von Pull Requests
- Deployen bei Releases
- Automatischen Labeln neuer Issues



Quelle: <https://docs.github.com/en/actions/about-github-actions/understanding-github-actions>

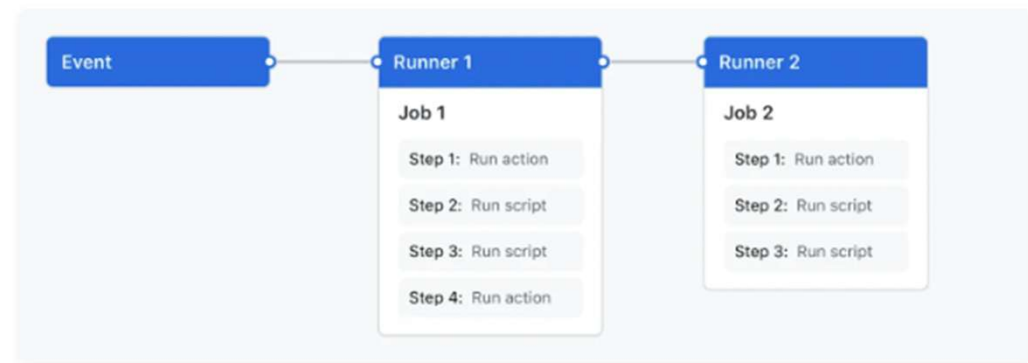
GITHUB ACTIONS KOMPONENTEN

GitHub Action Komponenten

2. Events

Ein **Event** löst einen Workflow aus. Mögliche Events sind z. B.:

- Ein **Commit** wird gepusht
- Ein **Pull Request** wird erstellt
- Ein **Issue** wird geöffnet
- Ein **geplanter Zeitablauf** (Schedule)



Quelle: <https://docs.github.com/en/actions/about-github-actions/understanding-github-actions>

GITHUB ACTIONS KOMPONENTEN

GitHub Action Komponenten

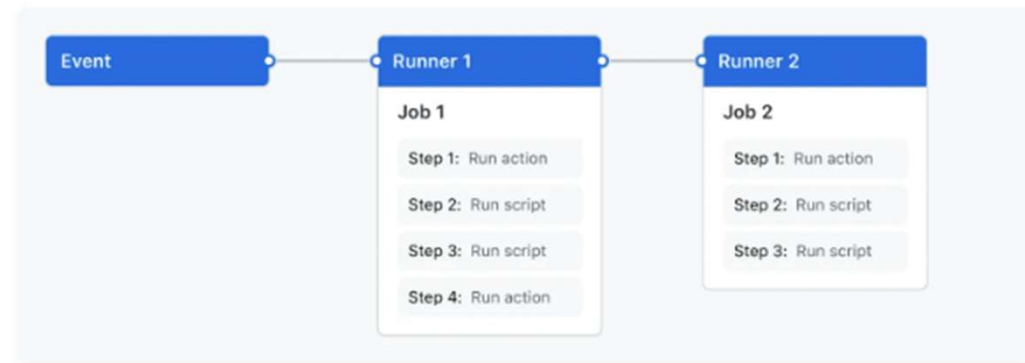
3. Jobs

Ein **Job** ist eine Sammlung von **Schritten (Steps)**, die auf dem gleichen **Runner** (virtueller Server) ausgeführt werden.

- Schritte können Skripte oder **wiederverwendbare Aktionen (Actions)** sein.
- Schritte laufen **nacheinander**, Jobs dagegen **standardmäßig parallel**.
- Man kann Abhängigkeiten zwischen Jobs definieren, sodass ein Job erst startet, wenn ein anderer abgeschlossen ist.

Beispiel:

Drei Build-Jobs für verschiedene Architekturen laufen parallel →
Danach ein Packaging-Job, der auf die Builds wartet.



Quelle: <https://docs.github.com/en/actions/about-github-actions/understanding-github-actions>

GITHUB ACTIONS KOMPONENTEN

GitHub Action Komponenten

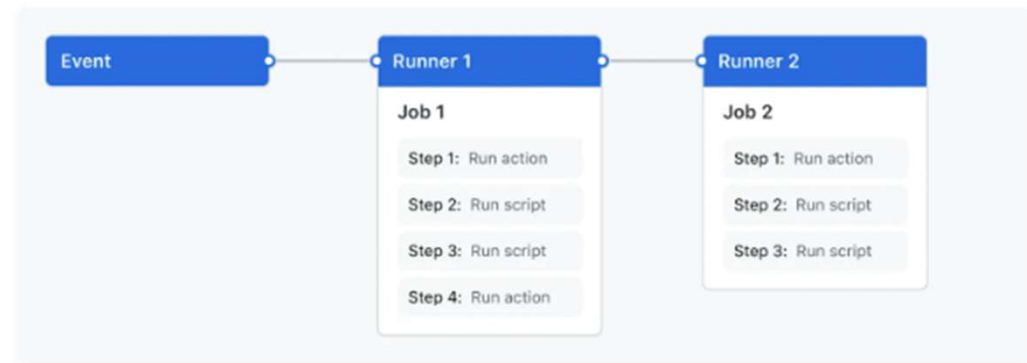
4. Actions

Eine **Action** ist eine wiederverwendbare Komponente, die komplexe, häufige Aufgaben automatisiert.

Beispiele:

- Git-Repo klonen
- Toolchain einrichten
- Authentifizierung beim Cloud-Provider vorbereiten

Du kannst eigene Actions schreiben oder bestehende aus dem **GitHub Marketplace** nutzen



Quelle: <https://docs.github.com/en/actions/about-github-actions/understanding-github-actions>

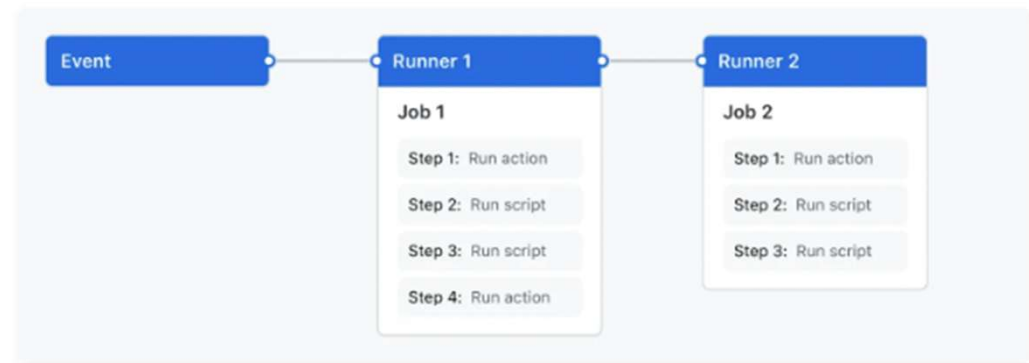
GITHUB ACTIONS KOMPONENTEN

GitHub Action Komponenten

5. Runners

Ein **Runner** ist ein Server, auf dem ein Workflow ausgeführt wird.

- GitHub stellt Runner mit Ubuntu, Windows und macOS bereit.
- Jeder Runner kann **einen Job gleichzeitig** ausführen.
- Für spezielle Anforderungen kannst du auch eigene Runner betreiben (self-hosted runners).



Quelle: <https://docs.github.com/en/actions/about-github-actions/understanding-github-actions>

GITHUB ACTIONS FÜR PYTHON

Anleitung:

<https://docs.github.com/en/actions/use-cases-and-examples/building-and-testing/building-and-testing-python>

Starter workflows:

<https://github.com/actions/starter-workflows/blob/main/ci/python-app.yml>

Integration / Delivery

20 Minuten Zeit



Aufgabe:

Erstellt einen workflow für eure Gebrauchtwagen-Applikation.

1. Findet euch in euren Zweier-Teams zusammen und erstellt einen **Continuous Integration workflow** mittels **Github Actions**.
2. Der **workflow** soll mindestens:
 - **dependencies** installieren
 - mit **Flake8** linten
 - die **Tests ausführen**, die ihr erstellt habt (unittest oder pytest)
 - Einen **test coverage report** in html ausgeben

